Course: LING285: Human Language and Technology
Title of Lecture: Lecture 11
Date: Tuesday, October 6
Professor: Khalil Iskarous

Section 1: One Mass, Two Directions

- A **mass-spring system** is a mass hooked up to a spring, and the whole system can be understood by asking what each part "wants."
  - **The spring wants to be at its equilibrium position**, which is its resting length. If it is stretched or squished, it pulls or pushes to get back there.
  - **A mass that is moving wants to keep on moving.** Once it gets going, it does not stop on its own.
  - Put these two wants together and the system vibrates. When the mass is pulled out and let go, the spring pulls it back toward equilibrium, but the mass is moving when it gets there, so it **keeps on moving** past equilibrium and squishes the spring. Now the spring pushes it back the other way, the mass overshoots again, and the cycle repeats.
  - In real life, friction from the table slowly brings the motion to a stop. The model leaves friction out, so in the model the system vibrates forever.
- The same system can be pictured in **two directions**, and it is important to be able to picture it both ways.
  - In the **longitudinal** picture, the motion is **left to right**, with the mass moving back and forth along the line of the spring.
  - In the **transverse** picture, the motion is **up and down**. The mass sits between two walls with a spring on each side, and when it is pulled up and let go, it goes up and down.
- **Both pictures move in exactly the same way**, and they do so for the same reason.
  - In the transverse picture, pulling the mass up **stretches the springs**, and because **springs hate being extended**, they pull the mass back down.
  - When the mass passes the middle, the springs are not stretched at all. But the mass is moving, so **it keeps on moving** and stretches them on the other side.
  - So the transverse system goes through stretch and release over and over, exactly like the longitudinal one. Some systems are easier to think about longitudinally and others transversely, but both describe the same vibration.
  - The two versions also vibrate at the same rate, because they have the same mass and the same total stiffness.

Section 2: Mass and Stiffness Change the Rate

- Two things set how fast a mass-spring system vibrates, and they are the **mass** and the **stiffness**. The number of back-and-forth cycles the system completes each second is called its **frequency**.
- **A heavier mass vibrates more slowly**, and this matches everyday experience. **Heavy branches sway back and forth slowly, while little branches sway back and forth quickly.**
  - A heavy mass is hard to get moving and hard to turn around, so each trip back and forth takes longer.
  - Making the mass much heavier therefore makes the whole system vibrate at a noticeably lower rate.

- **A stiffer spring makes the system vibrate faster.** Stiffness is harder to picture than mass, but the idea is that **the stiffer something is, the faster it wants to go back** to where it started.
  - A stiff spring pulls back hard, so it turns the mass around sooner.
  - Making the spring much stiffer therefore makes the whole system vibrate at a noticeably higher rate.
- The two rules point in **opposite directions**. **Raising the mass lowers the frequency, and raising the stiffness raises the frequency**, and these two rules explain everything else about modes.

Section 3: Two Masses, Two Modes

- With **two masses**, the system has **two walls**, a spring from each wall to a mass, and **one spring between the two masses**. All of the masses are equal and all of the springs are equal.
- This system can vibrate in **two different ways**, which are called **modes**, and **a mode is just another word for a way of vibrating.** The two modes are told apart by one question, which is whether the masses are going together or going against each other.
  - In **Mode 1**, the masses move **together**. In the transverse picture, both masses are pulled up and let go, and they go up and down side by side. In the longitudinal picture, both are pulled to the left and let go, and they move left and right as a pair.
  - In **Mode 2**, the masses move **against each other**. One mass is pulled down and the other up, and when they are let go, one goes up whenever the other goes down. In the longitudinal picture, one is pulled left and the other right, and they move toward each other and then away from each other.
- A system can also vibrate in **both modes at the same time**.
  - To start this, both masses are pulled up, but one is pulled higher than the other. Both start falling, which is Mode 1, but because they are uneven, the middle spring also pushes them against each other, which is Mode 2.
  - The result moves **together and against each other at the same time**, so sometimes both masses head the same way and sometimes they head opposite ways.
  - This seems strange, but real systems do it all the time, and speech production is one of them.
- **This foundation cannot be memorized**, because each step builds on the step before it. Trying to memorize it means memorizing pages without understanding them, while following the reasoning is what makes the later material make sense.

Section 4: Why the Second Mode Vibrates Faster

- The two modes have **different frequencies**. **When the masses go together, they move back and forth slowly, and when they go against each other, they move back and forth quickly.** So **Mode 2 has the higher frequency.**
- This is puzzling at first, because both modes use **the same two masses and the same three springs**, so nothing about the parts has changed. The explanation comes from asking **what each spring is doing** in each mode.
  - In **both modes**, the **two outer springs** stretch and compress as the masses move.

    - In **Mode 1**, the two masses move together, so the **middle spring never stretches or compresses**. It just rides along with the masses, acting less like a spring and more like a rigid bar, so **effectively Mode 1 has only two springs**.
    - In **Mode 2**, the masses move against each other, so the **middle spring is stretched and squished** over and over. **All three springs are working**, and all three hate being stretched.
- So Mode 2 **uses more stiffness**. **More stiffness means more pull back toward equilibrium, and more pull means a higher frequency.**
    - Nothing about the springs themselves changed, because what differs is **how each mode uses the springs**.
    - Mode 1 does not use the middle spring and Mode 2 does, and that one difference makes Mode 2 vibrate faster.
- The general rule follows from this. **The more the masses go against each other, the more the springs stretch and compress, so the more stiffness the mode uses and the higher its frequency.**

Section 5: Three Masses, Three Modes

- With **three masses**, the system has **three modes**, one more than with two masses. Each mode is a different way the masses can move relative to each other.
    - In **Mode 1**, all three masses go back and forth together, and this mode has the **lowest frequency**.
    - In **Mode 2**, the middle mass is left untouched while the two outer masses are pulled apart. The two outer ones move against each other, the **middle one does not move at all**, and this mode has the **middle frequency**.
    - In **Mode 3**, the middle mass moves **opposite to the two outer masses**, and this mode has the **highest frequency**. It is easiest to see in the transverse picture.
- The frequencies follow the same reasoning as with two masses.
    - **Mode 1 is very slow** because the masses move together, so the **two middle springs do nothing**. Their job is to stretch and compress, which is what raises frequency, and when they just ride along, they add no stiffness.
    - In **Mode 2**, the springs do stretch and compress, but not as much as they could.
    - In **Mode 3**, **everything is going against everything else**. That means **lots of stretching and lots of compression**, which uses the most stiffness, so Mode 3 moves fastest.
    - The masses and springs are the same in all three modes, so **only the way the masses move relative to each other changes the frequency.**
- All three modes can also happen **at the same time**, so the masses move together, against each other and middle against the sides all at once. However the system is started, it vibrates as some mix of its pure modes.
- Exciting some modes and not others is a real skill.
    - **Learning a musical instrument means learning how to make the masses and springs vibrate in certain modes and not others.** How a string is plucked or a violin is bowed decides which modes get excited, and these favored modes are heard as **notes**.
    - **A trained singer** does the same thing with the vocal folds, controlling their mass and stiffness to carry a tune.

- The same logic continues with **four masses**, which give four modes. All four can go together, the two on the left can go against the two on the right, the two in the middle can go against the two outer ones, or every mass can go against its neighbors.

Section 6: One More Mass, One More Mode

- The pattern holds for any number of masses, so **N masses give N modes.** One mass has only one way to vibrate, two masses have two, three masses have three, and so on.
  - Moving up to higher modes, **more parts move against each other**, so the pattern has **more reversals** in it.
  - More parts moving against each other means more stretching, so **higher modes have higher frequencies**.
  - In chains of separate masses like these, the mode frequencies are generally **not whole-number multiples** of the lowest frequency.
- Comparing the same mode across chains of different lengths reveals a shared pattern.
  - **Every Mode 1 is alike**, because all the masses go up and down together.
  - **Every Mode 2 is alike**, because the **left side goes against the right side**. With only two masses, that simply means one against the other.
  - **Every Mode 3 is alike**, because the **middle goes against the sides**.
- Packing in more and more masses and springs, all the way to an **infinite number**, gives a **string**, like a guitar or piano string. Every little piece of a string has mass, and every little piece stretches like a spring, so a string vibrates in all the same kinds of modes.
  - In the **lowest mode**, the whole string goes up and down together, with hardly any stretching between its pieces, and that mode gives the **lowest note**.
  - In **higher modes**, more and more parts of the string move against each other.
  - A string plucked without any skill vibrates in many modes at once.
- A system can also be **fixed on one side and free to sway on the other**. A **building** is like this, because it is fixed to the ground at the bottom and free to sway at the top in high winds or earthquakes.
  - Civil engineers design buildings to sway in **Mode 1**, with everything moving together, and never in higher modes, where floors would move against each other.
  - A building is not made so stiff that it cannot move at all. Instead it is designed to sway safely in its lowest mode.
  - The same vocabulary of modes describes music, buildings and, as the system builds up, speech.

Section 7: A String's Modes and Its Spectrum

- A **string fixed at both ends** vibrates in many modes. In the lowest mode the whole string goes up and down together, and each higher mode has more sections moving against each other and **completes more cycles in the same amount of time**. Frequency is measured in **Hertz**, which means **times per second**.
  - For a string fixed at both ends, each mode is a **whole-number multiple** of the lowest one, so the second mode is twice the lowest, the third is three times the lowest, and so on. Modes that line up this way are called **consecutive harmonics**.
  - The motion in the animation is slowed way down so that it can be followed by eye.

- A real string vibrates in **all of its modes at the same time**. Adding the separate motions together at every point gives one complicated, changing shape, and that combined motion is what an actual plucked string does.
- The **spectrum** of a sound tells **which frequencies are in it and how much of each there is**.
  - For a string moving in five modes at once, the spectrum has **five spikes**, one at the frequency of each moving mode.
  - If the string vibrated **only in Mode 3**, there would be no movement in the other modes, so the spectrum would have **just one spike, at the frequency of Mode 3**. That pattern is like a **one-hot encoding**, with one active slot and all the others at zero.
  - So the spectrum turns a complicated motion into a simple list of which modes are moving and how strongly.

Section 8: The Vocal Folds as a Membrane

- **The vocal folds are even more complicated than strings.** They are a bunch of muscle forming a surface, which is called a **membrane**, so their masses and springs are spread across two directions instead of along one line. The same logic of modes still applies to them.
  - In **Mode 1**, all of the masses go up and down together.
  - In **Mode 2**, the **left half goes against the right half**.
  - In **Mode 3**, the **middle is stuck** while the sides go against each other.
  - In **Mode 4**, four sections go against each other, and the patterns get more and more complicated in still higher modes.
- The frequencies given to the membrane's modes in the picture are assigned for teaching, not worked out for that rectangle. In real vocal folds, **all the modes happen at the same time**, so producing voice makes the vocal folds vibrate in many modes at once, and that creates a spectrum full of spikes.

Section 9: Theory, Prediction and Experiment

- **The scientific method** depends on **very clear statements about how systems work**, and these are usually written as **differential equations**, which describe how quantities change over time.
  - The simple spring equation is one example. Solving it predicts that if the spring is pulled it goes back, and if it is pushed it also goes back.
  - **Solving the equations gives a prediction** about how the world will behave **when it is actually measured**.
- For the voice, the reasoning moves from a theory to a prediction.
  - The theory is that **the vocal folds are a membrane** built from many masses and springs, and that when they vibrate, they do so in many modes at once.
  - The equations for the vocal folds are the same type as the spring equations, only more of them and **slightly more complicated**, so the intuition built from masses and springs is enough to reason with.
  - The prediction is that **if anyone's laryngeal vibration signal is measured, its spectrum will have evenly spaced spikes.** These evenly spaced spikes are called **harmonics**.

- **The prediction was tested in class.** Two students each made a sound with the **mouth wide open**, so that no particular vowel was being made, and the spectrum of each voice was examined in **Praat**, a program for analyzing speech.
  - In the first student's voice, the spikes came at a low frequency and its multiples, **evenly spaced** just as predicted.
  - In the second student's voice, the first spike was about twice as high, and the following spikes were again **evenly spaced**, only farther apart.
  - **The prediction held for both voices**, so a prediction built from simple masses and springs turned out to be precise and real.
- The difference between the two voices follows from the mass rule.
  - The second student's voice has a **higher pitch**, so their vocal folds are probably **smaller and thinner**.
  - **Thinner folds have less mass, and less mass means a higher frequency**, so the first harmonic is higher and every harmonic after it is spaced farther apart.
- This is part of how we **recognize people by their voices**. Each person's vocal folds produce their own spectrum, and becoming familiar with someone means becoming familiar with the spectrum of their voice. How a person says particular vowels also matters, but that depends on the vibration of the **air** above the vocal folds, not on the folds themselves.

Section 10: A String with One Fixed End and One Free End

- A string can also be **fixed at one end and free at the other**, just like the building that is fixed to the ground and sways at the top. This string is **fixed at the bottom** and **free at the top**.
  - The **fixed bottom** cannot move, and a point that never moves is called a **node**.
  - The **free top** moves the most, and a point of largest motion is called an **antinode**.
  - So every mode has to start with a node at the bottom and end with an antinode at the top.
- This changes which modes are possible. **Only the odd harmonics occur**, meaning one, three, five and seven times the lowest frequency. The modes at two times and four times the lowest are missing, because those patterns cannot end with free motion at the top.
  - **Mode 1** has no still point inside the string, so its motion grows steadily from the fixed bottom to the free top, like a building swaying as a whole.
  - Each higher mode adds more still points inside the string, with more sections swinging against each other.
  - When all four modes are excited equally, the spectrum measured at the free end has **four equal spikes**.
- Freeing one end also lowers the whole system. For a string of the **same length and the same wave speed**, switching from **fixed at both ends** to **fixed at one end and free at the other halves the lowest frequency**.
- The contrast between the two strings comes down to what happens at the ends.
  - When **both ends are fixed**, the string has **every whole-number multiple** of the lowest frequency.
  - When **one end is fixed and the other is free**, the string has **only the odd multiples**.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 12
Date: Tuesday, October 13
Professor: Khalil Iskarous

Section 1: Oscillation in the Economy and the 2026 Nobel Prize

- This year's **Nobel Prize in economics** went to work on the **microeconomic foundations of macroeconomic stabilization**. **Stabilization** means getting a system to **equilibrium and keeping it there**, which is the same **balancing feedback** a spring has, since a spring that is pulled or pushed always goes back to equilibrium.
- The prize was described as work on **boom-bust cycles**, the economy's swings between good times and bad times. In that picture, **unemployment and inflation go in reverse of each other**, rising and falling like two oscillating curves.
  - These are the same two curves found in a mass-spring system. One curve starts big, becomes negative and then becomes positive again, and that curve is **x**, the position. The other curve is its flip, which is **v**, the velocity.
  - In the economy, **inflation plays the part of x and unemployment plays the part of v**, so the math and the concept are exactly the same as for masses and springs, just in a completely different domain.
- The winning theory is called **Dynamic Stochastic General Equilibrium**, and each part of its name connects to masses, springs and gradient descent.
  - The word **dynamic** says that the theory describes how the economy **changes over time**.
  - The word **stochastic** says that the theory handles **shocks**, which are things that happen unpredictably, like a war starting or a factory burning down. This works like **stochastic gradient descent**, which gets knocked off course by one wildly off number but comes back to the average as it keeps learning, and in the same way the economic model returns toward balance after a shock.
  - The words **general equilibrium** say that the goal is an economy that **does not jump around** but settles into balance, and **stabilization policy** tries to **damp** the swings, making the oscillations smaller and smaller.
  - **Micro** refers to individual decisions, like buying something or applying for a job, while **macro** refers to whole-economy quantities like inflation and unemployment. Speech has the same structure, because many tiny oscillations in the mouth add up to a meaning that a listener understands without tracking each oscillation.
- **The same idea explains how economies work, how butterflies fly, how music works and how speech works.** Understanding one idea that runs across many domains is the kind of imagination that matters far more than memorizing.

Section 2: One Mass, Two Directions

- A mass-spring system rests on two facts, which are that **the spring wants to be at its equilibrium position** and that **a moving mass keeps on moving**. When the mass is pulled and let go, the spring pulls it back, the mass overshoots because it keeps moving, and so the system vibrates.
- The same system can be pictured **longitudinally**, with the mass moving **left and right**, or **transversely**, with the mass moving **up and down** between two springs. **Both pictures**

move in exactly the same way and at the same rate. Both views matter, because one will help with the vocal folds and the other will help with the air in the vocal tract.

Section 3: Mass and Stiffness Change the Rate

- **Raising the mass lowers the frequency, and raising the stiffness raises the frequency.**
  - A heavier mass is harder to turn around, so the system goes back and forth **fewer times per second**.
  - A stiffer spring pulls back harder, so the system goes back and forth **more times per second**.
  - Both rules hold whether the system is pictured longitudinally or transversely.

Section 4: Two Masses, Two Modes

- With two masses, the system can vibrate in **two modes**. A **mode** is a way of vibrating, and the two modes here are defined by whether the masses go **together** or **against each other**.
  - In **Mode 1**, both masses are lifted and let go, and they go up and down **together**.
  - In **Mode 2**, one mass is pulled up and the other down, and they go **against each other**.
  - When both are lifted but one higher than the other, **both modes happen at the same time**, so the masses go up and down together while also going against each other.
- **Mode 1 is slow and Mode 2 is fast**, even though both use the same masses and the same three springs. The difference is **how many springs are actually working** in each mode.
  - In **Mode 1**, the **middle spring never stretches or compresses**. It moves along like a rigid bar, so it is not behaving like a spring at all, and **effectively Mode 1 has only two springs.**
  - In **Mode 2**, **all three springs stretch and compress**, so **effectively Mode 2 has three springs.**
  - **More springs working means more stiffness, and more stiffness means a higher frequency**, because the stiffer the system is, the faster it has to go back to equilibrium.
- So **multiple masses give multiple modes, each with its own frequency.** The higher the mode, the more springs it uses, and the higher its frequency.

Section 5: Three Masses and Changing One Part

- With **three masses**, there are **three modes**.
  - In **Mode 1**, all three masses go back and forth **together**. The springs hardly stretch, so the stiffness in use is low and the frequency is the **lowest**.
  - In **Mode 2**, the **middle mass stays frozen** while the two outer masses go **against each other** around it. There is more stretching and compression, so the frequency is higher.
  - In **Mode 3**, **all three masses go against each other.** This mode has the **most stretching and compression**, so its frequency is the **highest**.
- Changing a single part of the system affects each mode differently, depending on what that part is doing in that mode.
- **Making the middle spring stiffer** affects only the modes that actually stretch it.
  - **Mode 1 stays the same**, because in Mode 1 the middle spring never stretches or compresses, so its stiffness does not matter.

    - **Mode 2 rises**, because in Mode 2 the middle spring is stretched and squished, so a stiffer spring pulls harder and the mode speeds up.
    - The more stiffness is packed into that one spring, the more Mode 2 rises, while Mode 1 never changes at all.
- **Making the middle mass heavier** affects only the modes in which that mass moves.
    - **Mode 1 goes down**, because the middle mass moves in Mode 1, so making it heavier slows the system.
    - **Mode 2 stays the same**, because the middle mass is frozen in Mode 2, and **a mass that does not move has no effect on frequency**, no matter how heavy it gets.
    - **Mode 3 goes down, and it goes down more than Mode 1 does**, because the middle mass moves even more in Mode 3 than in Mode 1, and **the more a mass moves, the more its weight affects the frequency**.
- The general rule is that **a mass's influence on frequency depends on how much it moves.** If it does not move at all it has no effect, if it moves a little it has a little effect, and if it moves a lot it has a big effect. These principles cannot be memorized as a list, because each one is worked out by asking what the part is doing in that mode.

Section 6: One More Mass, One More Mode, and the String

- Adding masses keeps the same patterns, so **one mass has one mode**, two masses have two, three have three, and so on. The modes repeat the same ideas as the chain gets longer.
    - **Every Mode 1 is the same idea**, with everything going up and down together.
    - **Every Mode 2 divides the system into two parts**, one going up while the other goes down, and if the number of masses is odd, the middle one is frozen.
    - **Every Mode 3 divides the system into three parts**, with the sides going up while the middle goes down.
    - Lower modes go up and down slowly because the system is not using much stiffness there, and higher modes go faster because more springs are stretching and compressing.
- Packing in more and more masses and springs gives a **string**, like a guitar or piano string, because a string is an **infinite number of masses and springs**. Every little piece of a string has weight, which is its mass, and every little piece hates being stretched or compressed, which is its springiness.
    - In the **first mode**, the whole string goes up and down together.
    - In the **second mode**, the two halves go against each other.
    - In the **third mode**, the middle goes against the sides.
    - In musical instruments, these mode frequencies are heard as **notes**.
- A real string vibrates in **many modes at the same time**, so its motion is the **sum of all the modes**. Hearing just a single mode would be a very annoying sound called a **pure tone**, like an ambulance siren, because real sounds have many modes together.
- **The ear can tell many instruments apart at once** because the ear is itself made of masses and springs that are well matched to the masses and springs vibrating in the world.

Section 7: Why the String's Frequencies Are Multiples

- The modes of a **string fixed at both ends** are **harmonics**, which means they are **exact multiples of the base frequency**, the frequency of the lowest mode.
- The multiples come from **how many copies of the first mode fit into each higher mode.**

    - The **first mode** is a single bump that goes up and down.
    - The **second mode** fits **two** of those bumps, one going up while the other goes down, so it vibrates **twice as fast**.
    - The **third mode** fits **three** bumps, so it vibrates **three times as fast**, and the pattern continues the same way for every higher mode.
    - So **the frequency of each mode is the base frequency multiplied by how many times the first mode fits into the whole string.**
- The **spectrum** tells **which frequencies a vibration has and how much amplitude is in each one**, with frequency running along one axis and amplitude, the amount of vibration, along the other.
    - This string's spectrum has a spike at each harmonic, because all of the modes are added together.
    - Because many frequencies happen at once, the actual motion looks **erratic**, much like the waveform of speech.
    - Measured at one marked point, the spikes come out **equal**, because each mode's strength was set to give the same size at that point.

Section 8: Movement and Stretching Alternate Along a String

- On a vibrating string, **the places with the most movement and the places with the most stretching are not the same places.**
    - In the **lowest mode**, the **most movement is in the middle**, where the string just goes up and down without being stretched.
    - The **most stretching is at the sides**, near the fixed ends, where the string barely moves at all.
- In higher modes, the same alternation repeats over and over along the string.
    - A stretch with **lots of movement and no stretching** is followed by a stretch with **lots of stretching and no movement**, and then by **lots of movement** again, and so on along the string.
    - So **where there is the most movement there is the least stretching, and where there is the most stretching there is the least movement.**
    - This alternation does not have to be memorized, because it can be worked out by watching what each piece of the string is doing.

Section 9: Principles of Mass-Spring Systems

- Everything so far can be summed up as a short list of principles, and each one comes from the same two facts, that mass resists being turned around and that springs hate being stretched.
- **Raising the mass makes the frequency go down, and raising the stiffness makes the frequency go up.** One line of the printed list says that stiffness lowers frequency, but that line is a typo, and the stiffness principle further down the same list correctly says that stiffness raises frequency.
- **A system of N masses has N modes of vibration, and each mode has a higher frequency than the one before.** Two masses can go together or against each other, so they have two modes, and three masses have three.
- **Increasing a mass lowers frequency in proportion to how much that mass moves, and how much it moves depends on where it is.**

    - A mass that does **not move** has **no effect**, however heavy it gets. At the very edges of a string there is no motion, so adding mass there changes nothing.
    - A mass that **moves a lot** lowers the frequency **a lot**. In the middle of the first, third and fifth modes there is lots of motion, so adding mass there drops the frequency a lot.
- **Where there is maximal movement there is no stretching, and where there is maximal stretching there is no movement.**
- **Increasing a spring's stiffness raises frequency in proportion to how much that spring stretches and compresses**, which also depends on where it is.
    - Where there is **no stretching or compression**, stiffening the spring has **no effect**.
    - Where there is **lots of stretching and compression**, stiffening it raises the frequency **a lot**.
- **In a system fixed at both ends, the harmonics are whole-number multiples of the base frequency**, because the base mode fits a whole number of times into each successive mode.
- **In a system fixed at one end and free at the other, the harmonics are odd multiples of the base frequency**, because the base mode fits an **odd number of times** into each successive mode.

Section 10: One Local Change, Two Different Effects

- On a string, one small section is marked where **Mode 4 is moving a lot** while **Mode 5 has no motion at all but lots of stretching**. Raising **both the mass and the stiffness** of that section has a different effect on each mode, and each effect has to be reasoned out separately.
- **For Mode 4**, the section is mostly moving.
    - **Raising the mass lowers the frequency**, because the mass is moving there.
    - **Raising the stiffness has no effect**, because the string there is just moving up and down without stretching or compressing.
    - So **the frequency of Mode 4 goes down**.
- **For Mode 5**, the section is mostly stretching.
    - **Raising the mass has no effect**, because there is no motion there.
    - **Raising the stiffness raises the frequency**, because there is lots of stretching there.
    - So **the frequency of Mode 5 goes up**.
- **Increasing both the stiffness and the mass at that one spot lowers the frequency of Mode 4 and raises the frequency of Mode 5.** The same principles get applied again and again. **Where there is no motion, added stiffness raises frequency, and where there is motion, added mass lowers frequency.**
- This method of changing one small part and predicting the effect is called **perturbation**. It was first worked out with **drums**, which are also an infinite number of masses and springs, by a scientist who added **wax**, and so added mass, to different parts of a drum.
    - Adding wax where the drum vibrated the most made the frequency **drop**.
    - Adding wax where there was no motion had **no effect** on the frequency.
    - He heard these differences with his own ears, long before cameras could capture the motion.

Section 11: The Vocal Folds as a Membrane

- **The larynx is a whole membrane**, a surface built from many masses and springs, a bit like a **drum**. It has many modes of vibration, and they follow the same patterns as before.
  - In **Mode 1**, all of the masses go up and down together.
  - In **Mode 2**, the left half goes against the right half.
  - In **Mode 3**, the middle is stuck while the sides go against each other.
  - The **higher modes** divide the surface into more and more parts.
- Just like the string, the membrane's modes are given frequencies that are multiples of the lowest one, and these values are assigned for teaching rather than worked out for that rectangle. Adding all of the motions together gives the combined surface, and its spectrum has **equal spikes**, one for each mode.

Section 12: Theory, Prediction and Experiment

- **The scientific method depends on very clear theories** about how the world works, which are usually written as **differential equations**. **Solving the equations gives a prediction** of what will be found when the world is actually measured.
- The theory and the prediction for the voice both follow from masses and springs.
  - The theory is that **the vocal folds are a membrane**, so when they vibrate, **their spectrum will have evenly spaced spikes**, which are called **harmonics**.
  - The differential equations are the same kind as before, only **slightly more complicated**.
  - The prediction is that **any laryngeal vibration signal will have a spectrum with evenly spaced harmonics.**
- **The experiment** checks this in **Praat** by taking a spectral slice of a voice.
  - The second spike comes out at **about double** the first, and the third comes out at **about triple** the first.
  - So the spikes come out **evenly spaced**, exactly as predicted. Every person's voice has its own base frequency, but the spikes are always multiples of it.
- The same reasoning explains how **smoking changes the voice**.
  - Heavy smokers can develop **nodules** on their vocal folds, and a nodule is like **adding a little mass** to one spot.
  - Even without a full nodule, smoking changes the **stiffness and mass of different parts** of the folds.
  - Those local changes shift different modes by different amounts, just like the wax on the drum, so **some harmonics stop being perfect multiples** of the base frequency and the voice sounds different.

Section 13: Fixed on One Side, Free on the Other

- A **building** is a system that is **fixed on one side and free on the other**, because it is fixed to the earth at the bottom while the top can sway.
  - Structural engineers design buildings to absorb high winds and earthquakes by swaying in **Mode 1**, with the whole building going back and forth together.
  - They design them **never to vibrate in higher modes**, where one floor would go one way and the next floor the other way, because that would be far worse.

    - Choosing the materials and structure of each floor so that the building favors only Mode 1 is a big part of structural engineering.
- **The air inside the vocal tract** is also fixed on one side and free on the other.
    - At the **bottom**, at the vocal folds, the **air cannot move**, because the vocal folds are closed and the air is stuck there.
    - At the **top**, at the lips, the air can move **in and out easily**.
    - Unlike a building, the air in the vocal tract **vibrates in all its modes**. Speaking changes the masses and springs of different parts of the tract, and that is how listeners know which vowel, consonant and word is being said.
- A system fixed at one end and free at the other has a **different spectrum**, because its frequencies jump by **odd multiples** of the base frequency instead of climbing through every multiple. The reason again comes from **how many copies of the first mode fit** into each higher mode.
    - The **first mode** goes from **no motion** at the fixed end to **lots of motion** at the free end.
    - The **second mode** fits **three** of those pieces that run from no motion to lots of motion, so it is **three times** the base frequency.
    - The **third mode** fits **five** of them, so it is **five times** the base frequency.
    - The pieces always come in **odd numbers**, because the system must always start with no motion at the fixed end and finish with lots of motion at the free end.
- The same holds for a string fixed at the bottom and free at the top, where **only the odd harmonics occur**. The fixed bottom is a **node**, a point with no motion, and the free top is an **antinode**, a point of largest motion. For the same length and wave speed, freeing one end **halves the lowest frequency**.

Section 14: Air in a Fixed-Free Tube

- The air in the vocal tract is modeled as a **mass-spring chain that is fixed at one end and free at the other**. The **masses** stand for the **air's inertia** and the **springs** stand for its **springiness**, and the chain is **fixed at the glottis** and **free at the lips**, with **identical masses** in a **uniform tube**.
- The air moves **lengthwise**, back and forth along the tube. When neighboring bits of air move toward each other, the air between them is **compressed**, and when they move apart, it **expands**.
- The first two modes matter most for vowels.
    - **Mode 1 corresponds to F1**, the **lowest air mode** of the tube. Its motion is still at the glottis and grows toward the lips.
    - **Mode 2 corresponds to F2**, the **next air mode**, and it vibrates **three times faster** than Mode 1.
    - **F1 and F2** are called **formants**, and they are the **resonances of the air in the vocal tract**.
- Three spots are marked along the tube, measured from the glottis toward the lips. The spot **one third of the way** up is labeled **/a/**, the spot **two thirds of the way** up is labeled **/i/**, and the **lip end** is labeled **/u/**.

Section 15: Vowels and Where the Tract Is Constricted

- Vowels differ in the **shape of the vocal tract**, which MRI images of five vowels seen from the side make visible. **High and low** describe tongue height, while **front, central and back** describe how far forward the tongue is, and together they place a **constriction**, which is a narrowing of the tract.
  - In **Dean**, the tongue is **high and front**, which makes a **constriction about two thirds of the way from the glottis**.
  - In **Dune**, the tongue is **high and back**, and the vowel has a **constriction at the lips**.
  - In **Done**, the tongue is **central**, and the tract has **no constriction** at all.
  - In **Don**, the tongue is **low and back**, which makes a **constriction about one third of the way from the glottis**.
  - In **Dan**, the tongue is **low and front**, and no constriction position is marked for it.
- These spots match the marks on the air chain, so each vowel's constriction lands on a different part of each air mode.

Section 16: /u/ and Loading at the Free End

- For **/u/**, the masses near the **lips get heavier**, which is called **loading the free end**.
- **Motion dominates at the lips in both modes**, because the lip end moves a lot in Mode 1 and also in Mode 2.
- Added mass where the air moves a lot slows the vibration, so **both F1 and F2 go down**.

Section 17: /i/ and a Constriction Near Two Thirds

- For **/i/**, the constriction is about **two thirds** of the way from the glottis.
  - In **Mode 1**, **the motion of the mass dominates** at that spot, so the added mass wins and **F1 goes down**.
  - In **Mode 2**, **springiness dominates** at that spot, so the added stiffness wins and **F2 goes up**.
- This is the same logic as Modes 4 and 5 on the string. One local change lowers one mode and raises another, because the spot **moves** in one mode and **stretches** in the other.

Section 18: /a/ and a Constriction Near One Third

- For **/a/**, the constriction is about **one third** of the way from the glottis.
  - In **Mode 1**, **springiness dominates** at that spot, so **F1 goes up**.
  - In **Mode 2**, **the motion of the mass dominates** at that spot, so **F2 goes down**.
- The result is **the reverse of /i/**. The rule never changes, since dominant stiffness raises a formant and dominant mass lowers it, and what changes which effect wins is where the constriction sits.
- Watching the air in the tube makes this visible. In the uniform tube for **schwa** and in the separate rows for **Mode 1, Mode 2 and Mode 3**, some places have air **moving a lot** and other places have air **compressing a lot**, and the two alternate along the tube. That alternation is why one constriction can push two formants in opposite directions.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 13
Date: Thursday, October 15
Professor: Khalil Iskarous

Section 1: Raising Mass and Stiffness Together

- The first principle is the simplest one. **As mass goes up, frequency goes down, and as stiffness goes up, frequency goes up.** A heavier mass is harder to turn around, so it vibrates more slowly, and a stiffer spring pulls back harder, so it vibrates faster.
- That principle by itself invites a **false argument**. Suppose something raises **both the mass and the stiffness** of a system. Since raising mass lowers frequency and raising stiffness raises it, it can seem that **the two effects cancel out** and nothing audible changes.
  - That reasoning is **only true of a system with a single mass**, where more mass and more stiffness really can cancel.
  - **In a system with many masses and springs, the effects do not cancel**, and the reason is a fact about where things happen along the system.
- **The places with the most motion are the places with the least springiness, and the places with the most springiness have the least motion.** So the location of a change decides which of the two effects counts.
  - At a spot on a string with **no movement but lots of stretching and compression**, making the spot heavier does **nothing**, because a mass only matters if it is moving. Making it stiffer **raises** the frequency, because that spot is stretching and compressing, so raising both there makes the frequency **go up**.
  - At a spot with **lots of movement but no stretching**, where the string just rides up and down, making the spot heavier **lowers** the frequency, because the mass is moving a lot. Making it stiffer does **almost nothing**, because the spring there is not stretching, so raising both there makes the frequency **go down**.
- It is even possible to raise both mass and stiffness and get **no effect at all**. That happens inside our own speech, at a constriction placed where neither effect matters.

Section 2: Principles of Mass-Spring Systems

- The rest of the principles build on the first one, and **they all interact with each other**. Each idea is simple, and each one follows from what masses and springs do.
- **A system with N masses has N modes of vibration, and each mode has a higher frequency than the last.** Two masses can go together or against each other, which gives two modes. Higher modes have higher frequencies because **more springs are active**, since more masses are going against each other and so the springs stretch and compress more.
- **Increasing a mass lowers frequency in proportion to how much that mass moves, and how much it moves depends on where it is.**
  - If the mass is **not moving**, increasing it has **no effect at all**.
  - If the mass is **moving a lot**, increasing it **lowers the frequency a lot**.
  - If the mass is moving only a little, the frequency drops only a little.
- **Increasing a spring's stiffness raises frequency in proportion to how much the spring stretches and compresses, and that also depends on where it is.**
  - Near places with **no motion**, there is lots of stretching and compression, so added stiffness has a **huge** effect on raising the frequency.

    - In places with **lots of motion**, there is no springiness in use, so added stiffness has **no** effect.
- **Where there is maximal movement there is no stretching, and where there is maximal stretching there is no movement.** This principle ties the mass principle and the stiffness principle together, because it decides where mass matters and where stiffness matters.
- The last two principles are about what happens at the ends of the system.
    - **In a system fixed at both ends, the harmonics are whole-number multiples of the base frequency**, because the base mode fits a whole number of times into each successive mode.
    - **In a system fixed at one end and free at the other, the formants are odd multiples of the base frequency**, because the base mode fits an **odd number of times** into each successive mode.

Section 3: One More Mass, One More Mode

- Chains with anywhere from **one mass up to six masses** all follow the same rule, which is that **N masses give N independent modes**. Each chain has **fixed ends, equal masses and equal springs**, and each row of the figure lays out every mode of its chain.
- Moving up to higher modes, **the pattern has more reversals and a higher vibration frequency.**
    - In the **lowest mode**, all the masses move together and the springs barely stretch, so the frequency is low.
    - In the **highest mode**, every mass moves against its neighbor, so every spring is working and the frequency is highest.
    - In chains of separate masses like these, the frequencies are generally **not whole-number multiples** of the lowest one.

Section 4: Harmonics and Why They Are Multiples

- The **vocal folds** are the main system in speech that is **fixed on its edges**. They sit right behind the Adam's apple and form a **membrane** of many masses and springs that is **fixed on all sides**. They vibrate in modes where the masses go up and down together, or where half go up while half go down, and so on into more complicated patterns.
- **The vibrations of a system fixed on its edges are called harmonics.**
    - The lowest one is the **base frequency**, which belongs to the lowest mode.
    - Each following harmonic is a **multiple** of the base, so the second harmonic is twice the base, the third is three times the base, and so on. Frequency is measured in **Hertz**, which means times per second.
    - The base frequency is different for different people, but whatever it is, the harmonics above it are always its multiples.
- The multiples come from **counting how many copies of the first mode fit into each higher mode.**
    - On a string fixed at both ends, the first mode is **half a wave**, with one section going up and down.
    - The **second mode** has **two** of those sections, one going up while the other goes down, so it vibrates twice as fast as the first.
    - The **fifth mode** has **five** sections, so it vibrates five times as fast as the first.

    - Each piece from one still point to the next is called a **half-wave packet**. The first mode holds one packet, the second holds two, and so on, so **higher harmonics fit more half-wave packets into the same length.**
- A **string fixed at both ends** vibrates in all of its modes **at once**, so its combined motion is the sum of all of them. Measured at one point, its spectrum has **equal spikes**, one for each mode, because each mode's strength was set to give the same size at that point.

Section 5: The Membrane Combines Modes

- A **membrane** is a flat surface **fixed on all of its edges**, and its modes are named with **two numbers** that count its moving regions in each direction.
    - In the **(1, 1)** mode, one region rises and falls as a whole.
    - In the **(2, 1)** and **(1, 2)** modes, two regions go against each other, along one direction or the other.
    - In the **(2, 2)** mode, four regions alternate with each other.
    - In the **(3, 1)** mode, three regions go against each other along one direction.
- The same counting works on the membrane. The first mode fits into the second mode **twice**, so the second vibrates twice as fast, and it fits into the fourth mode **four times**, so the fourth vibrates four times as fast. The frequencies given to the five modes in the picture are assigned for teaching rather than worked out for this rectangle. The **combined surface** is the exact sum of all five motions, and its spectrum at a pickup point has **five equal spikes**.

Section 6: Fixed-Free Systems and Formants

- A **building** is **fixed on one side**, at the ground, and **free on the other**, at the top.
    - Engineers want it to sway in **Mode 1**, all together, which is mildly annoying on a high floor but safe.
    - A higher mode, with **some floors going one way and others going the other way**, would shatter the structure.
- A system fixed at one end and free at the other has a **different counting rule**. When both ends are fixed, the basic piece runs from no motion, to lots of motion, and back to no motion. When one end is free, the basic piece runs only **from no motion to lots of motion**, with no still point at the free end.
    - The next mode up fits **three** of those pieces, so its frequency is three times the first.
    - The mode after that fits **five** of them, so its frequency is five times the first.
    - The pieces always come in **odd numbers**, so the frequencies are **odd multiples** of the first.
- **The frequencies of a system fixed at one end and free at the other are called formants**, which is a different name from harmonics.
    - **Formants are the frequencies of vibration of a system fixed on one side and completely free on the other.**
    - **Their major property is that they are odd multiples of the first formant**, so they go one, three, five, seven and nine times the first, and so on.
- **The big, important system of this kind in speech is the air in the vocal tract.** Its vibration makes all the vowels and consonants of speech, and speech works by **changing the stiffness and mass in different parts of the vocal tract**, which moves the formants.

Section 7: Quarter-Wave Packets in the Fixed-Free String

- A **string fixed at the bottom and free at the top** follows the rule for one fixed end and one free end.
  - The **fixed bottom** cannot move, so it is a **node**, which is a point with no motion.
  - The **free top** moves the most, so it is an **antinode**, which is a point of largest motion.
  - So **only the odd harmonics occur** on this string.
  - For the **same length and wave speed**, freeing one end **halves the lowest frequency**.
- The basic piece from a node to an antinode is called a **quarter-wave packet**, and each mode holds an **odd number** of them.
  - **Mode 1** holds just **one packet**, and **Mode 2** holds **three packets**.
  - **Mode 3** holds **five packets**, and **Mode 4** holds **seven packets**.
- The counts skip the even numbers because of what has to happen at the ends.
  - Adding just **one** packet would leave a **node** at the free top, and that is not allowed.
  - Adding **two** packets at a time keeps the **antinode** at the top.
  - So **one fixed end and one free end give only odd multiples**, while **two fixed ends give every multiple**.

Section 8: One fixed end and one free end

- A **building** is fixed on one side at the ground and free on the other at the top. Engineers want it to sway in the first mode, all together, which is mildly annoying on a high floor but safe. A higher mode, with some floors going one way and others going the other way, would shatter the structure.
- A fixed-free system has a **different counting rule**. A fixed-fixed system's basic piece goes from nothing to lots of motion and back to nothing, while a fixed-free system's basic piece goes from nothing to lots of motion, with no still point at the free end.
- If the first mode is 100 Hz, the next mode fits three of those pieces, so it is 300 Hz, and the third fits five, so it is 500 Hz.
- The frequencies of a fixed-free system are called **formants**, and their major property is that they are **odd multiples** of the first.
- The string fixed at the bottom and free at the top has a **node** at the fixed end, a point with no motion, and an **antinode** at the free end, a point of largest motion. Its four modes are at 100, 300, 500 and 700 Hz, so only odd harmonics occur.
- For the same length and wave speed, freeing one end halves the lowest frequency, so this string was retuned to keep its lowest mode at 100 Hz.
- The big, important fixed-free system in speech is the air in the vocal tract, whose vibration makes all the vowels and consonants.

Section 9: Why fixed–free frequencies are odd multiples

- The basic piece from a node to an antinode is a **quarter-wave packet**, and each fixed-free mode holds an odd number of them. The first mode holds one packet, the second holds three, the third holds five and the fourth holds seven.
- The counts skip the even numbers because of the ends. Adding one packet would leave a node at the free top, which is not allowed, but adding two packets keeps the antinode at the top.

- So one fixed end and one free end give only odd multiples, while two fixed ends give every multiple.
- In both cases, more packets in the same length means shorter waves and higher frequencies.

Section 10: The Air in the Vocal Tract as a Fixed-Free System

- The air in the vocal tract can be modeled as a **chain of masses and springs that is fixed at one end and free at the other**, the same as any other vibrating system. The **masses** stand for the air's inertia, its tendency to keep moving, and the **springs** stand for its springiness when squeezed. The chain is **fixed at one end**, where there is no motion, and **completely free at the other**, and while masses and springs are easier to see than air, the model and the air itself behave the same way.
- The first two modes are the ones that matter most.
  - **Mode 1 corresponds to F1**, the **lowest air mode** of the tube. Its motion is still at the fixed end and grows toward the free end.
  - **Mode 2 corresponds to F2**, the next mode up. It fits **three** of the pieces that run from no motion to lots of motion, while Mode 1 fits only one, so in a **uniform tube** Mode 2 vibrates **three times faster** than Mode 1.
  - The letter **F stands for formant**, so F1 is the first formant and F2 is the second.
- Three spots are marked along the tube, measured from the fixed end. The spot **one third of the way** up is labeled **/a/**, the spot **two thirds of the way** up is labeled **/i/**, and the **free end** is labeled **/u/**, and each spot takes part in both modes.

Section 11: Motion and Compression in the Air

- The air inside the mouth is **fixed at the vocal folds and free at the lips**.
  - At the **vocal folds**, the folds are closed, so the air **cannot go back and forth**. It is stuck there, which makes that end **fixed**.
  - At the **lips**, the mouth is open, so the air **goes back and forth freely**, which makes that end **free**. Air that goes out also comes back in, just as a mass goes back and forth.
  - Each blue circle in the figure stands for billions of air molecules.
- Two kinds of motion tell the masses and the springs apart.
  - Where neighboring lines of air **come really close together and then move really far apart**, the air is **stretching and compressing**, and that is a sign of **springiness**.
  - Where the lines of air **just move back and forth together** without their spacing changing, the air is acting as a **mass**.
  - The easiest place to see pure mass is the **free end at the lips**, because the air there just moves back and forth with no compression at all.
- **Mode 1 of the air is the same idea as a building swaying in Mode 1.**
  - There is **no motion at the bottom** but **lots of compression and expansion** there, just as a building's base is fixed while its structure stretches and compresses down low.
  - There is **lots of motion at the top** but **no compression** there, just as the top of a building sways freely without being stretched.
- **Higher modes have more fixed points**, which are places with no motion at all.
  - **Mode 1** has only **one** fixed point, which is at the vocal folds.
  - **Mode 2** has **two**, one at the vocal folds and one partway up the tract.
  - **Mode 3** has **three**, and the pattern continues the same way.

    - Around each point with no motion there is lots of compression and stretching, so motion and springiness **alternate** along the tract, just as they do on a string.
- **The formants follow the rule of odd multiples**, so the second formant is three times the first and the third formant is five times the first. The actual value of the first formant depends on **how heavy the air is and how springy it is**.
- The rule holds in real speech.
    - A recording of **schwa**, the vowel in "hut," was measured, and its three formants came out very close to the one, three and five times pattern that the rule predicts.
    - So **a theory built from pure intuition about masses and springs predicts real measurements.**

Section 12: Vowels and Constrictions

- Vowels differ in **where the air is squeezed** in the vocal tract. A **constriction** is a place where the tongue or lips narrow the passage and **squeeze the air**.
    - **Done** has **no constriction**, so the airflow is **unimpeded** everywhere and the tract is open evenly along its length.
    - **Dean** is a **high front** vowel, and it squeezes the air under the roof of the mouth, **two thirds of the way from the glottis**. The **glottis** is the tiny space between the vocal folds, at the fixed end.
    - **Don** is a **low back** vowel, and it squeezes the air **one third of the way from the glottis**, far in the back.
    - **Dune** is a **high back** vowel that also has **rounding**, with the lips very close together.
    - **Dan** is a vowel that is **low and front**.
- **The vowel /u/ has two constrictions**, one made with the tongue far in the back and one made at the lips.
    - The **constriction at the back has no effect on any of the formants**, even though it is large, because raising mass and stiffness there changes nothing given where it falls in the modes.
    - So the lip rounding is the part of /u/ that actually changes the formants.

Section 13: What Constricting Air Does

- **Constricting air makes it stiffer.** Squeezing air is like pumping up a tire, because the more the air is squeezed, the **stiffer** the tire gets.
- **Constricting air also makes it more massive.** Squeezing air into a smaller volume raises its **density**, and **the smaller the volume, the higher the density**, so constricted air is effectively **heavier**.
- **So a constriction raises both the effective mass and the effective stiffness.** The false conclusion is that the two cancel, but **they do not**, because speech is full of changing frequencies even though all a speaker does is raise mass and stiffness in different parts of the tract.
    - The real answer comes from **location**. At a point with **no motion but lots of stretching and compression**, the added mass does **nothing** and the added stiffness **raises** the frequency.
    - At a point with **lots of motion and no compression**, the added mass **lowers** the frequency and the added stiffness does **nothing**.

- So whether a formant goes up or down depends on **whether its mode is moving or compressing at the constriction**.

Section 14: Vowel shape locates a constriction

- Vowels differ in **where the air is squeezed** in the vocal tract. A **constriction** is a place where the tongue or lips narrow the passage and squeeze the air.
- Done has no constriction, so the airflow is unimpeded everywhere.
- Dean, a high front vowel, squeezes the air under the roof of the mouth, two thirds of the way from the **glottis**, the tiny space between the vocal folds at the fixed end.
- Don, a low back vowel, squeezes the air one third of the way from the glottis, far in the back, while Dune, a high back vowel, also has rounding with the lips very close together, and Dan is low and front.
- **/u/ has two constrictions**, one with the tongue far in the back and one at the lips. The constriction at the back has no effect on any of the formants, even though it is large, because of where it falls in the modes.
- So the lip rounding is the part of /u/ that actually changes the formants.

Section 15: Why a constriction can raise one formant and lower another

- **Constricting air makes it stiffer**, just as squeezing the air in a tire makes the tire stiffer. **Constricting air also makes it more massive**, because squeezing air into a smaller volume raises its density, and the smaller the volume, the higher the density.
- So a constriction raises both the effective mass and the effective stiffness of the air.
- The false conclusion is that the two effects cancel. They do not, since speech is full of changing frequencies even though all a speaker does is raise mass and stiffness in different parts of the tract. The real answer comes from location.
- At a point with no motion but lots of stretching and compression, the added mass does nothing and the added stiffness raises the frequency.
- At a point with lots of motion and no compression, the added mass lowers the frequency and the added stiffness does nothing.
- So whether a formant goes up or down depends on whether its mode is moving or compressing at the constriction.

Section 16: /u/ and Loading the Free End

- For **/u/**, the masses at the **free end get heavier**, which is called **loading the free end**. Rounding the lips squeezes the air between them, which raises its mass and its stiffness.
- **Both modes are all motion at the free end of the tube.** The air there moves back and forth a lot in Mode 1 and in Mode 2, with **hardly any compression or expansion**.
  - So the **added mass** matters, because the air there is moving.
  - The **added stiffness** does not matter, because there is no compression there.
  - Added mass where there is motion **lowers frequency**, so **both F1 and F2 go down**.
- **A recording confirms the prediction.** Starting from schwa, a student moved into /u/, and **both the first and second formants dropped**, because the increased mass, or density, of the air between the lips lowered them.

Section 17: /u/ loads the moving lip end

- For **/u/**, the masses at the free end get heavier, which is **free-end loading**. Rounding the lips squeezes the air between them and raises its mass and stiffness. **At the free end, both modes are all motion**, with hardly any compression or expansion, so the added mass matters and the added stiffness does not.
- Added mass where there is motion lowers frequency, so F1 goes down and F2 goes down.
- **A recording confirms the prediction.** Starting from schwa, with formants near 500, 1500 and 2500, a student moved into /u/, and both the first and second formants dropped.
- The increased density of the air between the lips is what lowered them.

Section 18: /i/ and a Constriction at Two Thirds

- For **/i/**, the air is squeezed **two thirds of the way from the fixed end**, and at that spot the two modes are doing different things.
  - In **Mode 1**, there is **lots of motion** and hardly any compression, so the spot is **dominated by mass**.
  - In **Mode 2**, there is **lots of compression and expansion**, so the spot is **dominated by stiffness**.
- The effects follow directly from the principles.
  - **F1 goes down**, because added mass where there is motion lowers the frequency.
  - **F2 goes up**, because added stiffness where there is compression raises the frequency.
  - So one constriction pushes **F1 down and F2 up** at the same time.
- **A recording confirms this prediction too.** Starting from schwa and moving into /i/, **the first formant dropped a lot and the second formant went up.**
- **Speech is masses and springs from start to finish.** Speakers control masses and springs to make the **vocal folds** vibrate, that vibration drives the **air in the vocal tract**, whose masses and springs are shaped by the tongue and lips, and the sound then travels into the listener's **ear**, which also has a structure of masses and springs.

Section 19: /i/ has opposite effects on F1 and F2

- For **/i/**, the air is squeezed two thirds of the way from the fixed end, and at that spot the two modes are doing different things. In the first mode there is lots of motion and hardly any compression, so the spot is mass-dominated. In the second mode there is lots of compression and expansion, so the spot is stiffness-dominated.
- So F1 goes down, because added mass where there is motion lowers the frequency.
- And F2 goes up, because added stiffness where there is compression raises the frequency.
- **A recording confirms this as well.** Starting from schwa and moving into /i/, the first formant dropped a lot and the second went up.

Section 20: /a/ and a Constriction at One Third

- For **/a/**, the constriction is **one third of the way from the glottis**, which is closer to the fixed end.
  - In **Mode 1**, **springiness dominates** at that spot, so the added stiffness wins and **F1 goes up**.
  - In **Mode 2**, **the motion of the mass dominates** at that spot, so the added mass wins and **F2 goes down**.

- The result is **F1 up and F2 down**, which is the **reverse of /i/**. The same rule decides both cases, since stiffness where the air compresses raises a formant and mass where the air moves lowers it.

Section 21: /a/ reverses the two effects

- For **/a/**, the constriction is one third of the way from the glottis, closer to the fixed end. In the first mode springiness dominates at that spot, so the added stiffness wins and F1 goes up. In the second mode mass motion dominates at that spot, so the added mass wins and F2 goes down.
- The result is F1 up and F2 down, the reverse of /i/.
- The same rule decides both cases, since stiffness where the air compresses raises a formant and mass where the air moves lowers it.

Section 22: Source and Filter Together

- In **source-filter theory**, a vowel comes from two systems working at the same time.
  - The **source** is the **glottal vibration** of the vocal folds. **The glottal source gives a harmonic comb**, which is a set of evenly spaced harmonics, and **before the vocal tract** acts on them, they are just **evenly spaced lines of equal height**.
  - The **filter** is the air in the vocal tract, a system fixed at one end and free at the other with its own formants, and **the vocal tract filter sculpts the heights of those harmonics.**
- In the spectrum of real speech, **both systems appear at once**.
  - **The harmonics are all still there**, at the same frequencies that the vocal folds produce.
  - **The amplitudes of the harmonics are shaped by where the formants are**, so harmonics near a formant come out **high** and harmonics far from any formant come out **low**.
  - For **schwa**, the formants are evenly spread, so the harmonics bunch into tall peaks at evenly spaced spots up the spectrum.
- **The harmonics stay the same while their heights change, and the outline of those heights is what identifies the vowel.**
  - The **envelope** is the outline traced by the tops of the harmonics.
  - **Different vowels put their formants in different places**, so the same harmonics get shaped differently.
  - For **/i/**, F1 is low and F2 is high, so the tall harmonics cluster near the bottom and again high up, with a gap between them.
  - For **/a/**, F1 is higher and F2 is lower, so the tall harmonics gather in one broad group lower down.
  - Changing the vowel never moves the harmonics, because it only reshapes their heights by moving the formants.

Section 23: Spectrum Versus Spectrogram

- **A spectrum captures only one instant**, with frequency along one axis and amplitude along the other. But speech is **time-varying**, because constrictions are made and unmade as the tongue and lips move, so one spectrum is not enough.
- **A spectrogram stacks many spectra over time.**

    - **Time runs along the horizontal axis**, and **frequency runs up the vertical axis**.
    - **Amplitude is shown as darkness**, so high amplitude is **black**, medium amplitude is **gray**, and low amplitude is **white**.
    - So a formant appears as a **dark band** at its frequency.
- In a spectrogram of **/ia/**, the dark bands follow the formants over time.
    - During **/i/**, **F1 is very low and F2 is very high**, so there is a dark band low down, a gap, and another dark band high up, with the two formants **far apart**.
    - Moving into **/a/**, the formants **come close together**, as the low band rises and the high band falls.

Section 24: The Problem of Harmonics and Formants

- **Formants can only be seen through the harmonics they shape**, so the spacing of the harmonics matters a great deal.
    - When the harmonics are closely spaced, each formant has **lots of harmonics to shape**, so each formant band is easy to see.
    - When the harmonics are widely spaced, far fewer of them fall near each formant, and **it becomes very hard to tell harmonics from formants.**
- Holding the vocal tract filter fixed and changing only **F0**, the base frequency of the vocal folds, makes this clear.
    - A **low F0** gives **dense** harmonics that trace each formant band well.
    - A **higher F0** gives **sparser** harmonics, so a formant can fall **between** them.
    - A **very high F0** gives so few harmonics that the formants may look like **isolated harmonics** instead of bands.
    - So **the formants are a smooth envelope and the harmonics are samples of it, and a higher F0 gives fewer samples.** Raising F0 does not move the formants, but it leaves fewer harmonics to reveal them.
- High voices make this a real problem for listeners too.
    - **A toddler's harmonics are widely spaced**, so it is hard to tell where their formants are unless the listener is used to how that toddler speaks.
    - **A soprano's first harmonic is very high**, and with so few harmonics near the formants, most people cannot follow the words at an opera, even while watching the singer's lips.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 14
Date: Tuesday, October 20
Professor: Khalil Iskarous

Section 1: The Vocal Folds and F0

- Speech has **two major vibrating systems**, and the first is the **vocal folds**. They are a **membrane fixed on all sides**, made of a whole bunch of masses and springs, and they vibrate in many modes.
  - In **Mode 1**, all the masses go up and down together.
  - In **Mode 2**, half of them go up while the other half go down.
  - Higher modes split the membrane into more and more parts going against each other.
  - This is literally what happens in the throat during speech, and measuring the spectrum of real voices confirms it.
- Because the vocal folds are **fixed on all sides**, their frequencies are **harmonics**, which are **whole-number multiples** of the lowest one. **The lowest harmonic is called F0.** Each person has a different F0, but every voice has its harmonics stacked at multiples of its own F0.
- A membrane is easy to break down into modes, and its modes are named by **how many moving regions it has in each direction**, such as **(1, 1)** for one region and **(2, 2)** for four regions. The frequencies given to its modes in the picture are assigned for teaching rather than worked out for the membrane. Added together, the separate motions make one complicated surface, and its spectrum at a pickup point has **equal spikes**, one for each mode.

Section 2: A String with One Fixed End and One Free End

- The second major vibrating system is fixed differently, so it follows a different rule, and a **string fixed at the bottom and free at the top** is the simplest version of it.
  - At the **fixed bottom**, the string cannot move, so that point is a **node**.
  - At the **free top**, the string moves the most, so that point is an **antinode**.
  - So every mode must have a node at the bottom and an antinode at the top.
- Because of those ends, **only the odd harmonics occur**, at one, three, five and seven times the lowest frequency. With equal excitation of each mode, the spectrum at the free end has **equal spikes** at those odd multiples.
- For a string of the **same length and wave speed**, switching from **fixed at both ends** to **fixed at only one end halves the lowest frequency**.

Section 3: Quarter-Wave Packets

- A **quarter-wave packet** is the piece of a wave that runs from a **node to an antinode**. That piece is exactly what joins the fixed bottom to the free top, so each mode is built out of these packets.
  - **Mode 1** holds just **one packet**, so it vibrates at the lowest frequency.
  - **Mode 2** holds **three packets**, so it vibrates three times as fast as Mode 1.
  - **Mode 3** holds **five packets**, and **Mode 4** holds **seven packets**, so they vibrate five and seven times as fast.

- The counts are always **odd**, because the string must still end with an antinode at the top. Adding one packet would leave a node there, so packets get added **two at a time**, and **more packets in the same length means shorter waves and faster vibration.**

Section 4: The Air in the Vocal Tract

- The second vibrating system is the **air in the vocal tract**. Picture the space inside the mouth and throat straightened out into a tube, with the flesh of the tongue, the palate and the throat forming its walls.
  - The air is **fixed at the glottis**, the space between the vocal folds, because the folds are closed and the air cannot move back and forth there.
  - The air is **free at the lips**, where it can go **in and out** over and over.
  - So the air in the vocal tract is fixed at one end and free at the other, and its frequencies are **formants** that fall at **odd multiples**.
- The air can be modeled as a **mass-spring chain that is fixed at one end and free at the other**, with **identical masses** in a **uniform tube**.
  - **Mode 1 corresponds to F1**, the **lowest air mode** of the tube.
  - **Mode 2 corresponds to F2**, the **next air mode**, and it vibrates **three times faster** than Mode 1.
  - Three spots are marked along the tube, measured from the glottis. The spot **one third of the way** up is labeled **/a/**, the spot **two thirds of the way** up is labeled **/i/**, and the **lip end** is labeled **/u/**.

Section 5: Where the Air Is Springy and Where It Is Massy

- The air does **two different things** at different places, and both can be seen directly.
  - Where the slivers of air **come very close together and then move far apart**, the air is **compressing and expanding**, so it is acting as a **spring**. That happens at the **closed end**, because air can be squeezed against something closed.
  - Where the slivers stay **equally spaced** and just **move back and forth**, the air is acting as a **mass**. That happens at the **open end**, because there is nothing there to squeeze against.
- In **Mode 1**, the air is **very springy at the closed end**, then less and less springy and **more and more like a mass** toward the open end. In **higher modes**, springiness and mass **alternate** along the tube, and they repeat more times the higher the mode is.
- **In every mode, the air at the open end acts like a mass**, going back and forth with hardly any expansion, and **the air near the closed end always acts like a spring.**
- **Constricting the tract at a springy spot raises the formant of that mode.** Take a spot in the **third mode** where the air is bunched up and expanding, so it is very springy, and reason through what a constriction does there.
  - **A constriction always increases both mass and stiffness**, because squeezing air makes it stiffer, the way squeezing a tire does, and bunching it up makes it effectively heavier.
  - At this spot the air is **acting like a pure spring**, with no back-and-forth movement and only compression and expansion.
  - So the **mass does not matter** there, and only the **added stiffness** counts, and since added stiffness raises frequency, **F3 goes up**.

- This chain of reasoning has to be built fresh at each spot, because the situation is slightly different everywhere, so following the logic step by step matters more than memorizing the answer for one spot.

Section 6: Vowels and Where the Air Is Squeezed

- Real images of speaking come from an **MRI** that films the vocal tract during speech, and the images reveal **where each vowel squeezes the air**.
  - **High** and **low** describe how high the tongue is, while **front**, **central** and **back** describe how far forward it is.
  - In **Dean**, the tongue is **high and front**, which makes a constriction **two thirds of the way from the glottis**.
  - In **Dune**, the tongue is **high and back**, and the vowel has a constriction **at the lips**.
  - In **Done**, the tongue is **central**, and the tract has **no constriction** at all.
  - In **Don**, the tongue is **low and back**, which squeezes the air **far back in the throat, one third of the way from the glottis**.
  - In **Dan**, the tongue is **low and front**.
- **Combining these images with the model** is where theory meets reality. The model treats the air as masses and springs that are fixed on one side and free to move on the other, and a constriction is drawn as **stiffer springs and heavier masses** at that spot. Where each vowel puts its constriction decides which formants go up and which go down.

Section 7: What a Constriction Does to Mass and Stiffness

- **Compressing air raises its stiffness**, so **constricting at a place with lots of compression raises the frequency**.
- **Bunching up air raises its mass, or density**, so **constricting where the air moves a lot adds mass**, and that **lowers the frequency**.
- **So making a constriction increases both mass and stiffness**, but that does **not** mean the effects cancel and constrictions leave frequency unchanged. **The parts of a vibrating medium that move the most alternate with the parts that compress and uncompress the most**, so at any given spot in a mode, one effect wins.
  - If the air there is mostly **moving**, the added **mass** wins and the formant **goes down**.
  - If the air there is mostly **compressing**, the added **stiffness** wins and the formant **goes up**.
  - The same constriction can do opposite things to different formants, because each mode has its own pattern of motion and compression.

Section 8: /u/ and Loading the Free End

- For **/u/**, the **lips squeeze the air** at the free end, which is called **loading the free end**, and that raises both the mass and the stiffness of the air at the lips.
- **The air at the open end acts like a mass in every mode.** It goes back and forth with hardly any squeezing, because there is nothing there to squeeze against.
  - So the added **stiffness does not matter**, because there is no compression or expansion there.
  - The added **mass does matter**, because the air there is moving a lot.
  - **Increasing mass where the air moves lowers the frequency**, so **both F1 and F2 go down**.

- This is what happens in the mouth for vowels like the ones in **dune, moon and through**, and measurements match it. Starting from schwa, a student's **/u/** had an **F1 far below the F1 of schwa**.

Section 9: /i/ and a Constriction at Two Thirds

- For **/i/**, the air is squeezed **two thirds of the way from the closed end**, and the two modes are doing different things at that spot.
  - In **Mode 1**, the air there is **mostly moving back and forth**, so it acts like a **mass**.
  - In **Mode 2**, the air there is **mostly compressing**, so it acts like a **spring**.
- **A constriction always raises both mass and stiffness**, so the outcome depends on which one matters in each mode.
  - **F1 goes down**, because the air is moving and the added mass wins.
  - **F2 goes up**, because the air is compressing and the added stiffness wins.
  - So **wherever stiffness is what matters, the frequency goes up, and wherever mass is what matters, the frequency goes down.**
- Moving from schwa into /i/ in a recording, **F1 drops and F2 rises**, exactly as predicted. If this were not real, no one could tell what anyone else was saying.

Section 10: /a/ and a Constriction at One Third

- For **/a/**, the air is squeezed **far back, about one third of the way from the closed end**.
  - In **Mode 1**, that spot is **close to the closed end**, and **near the closed end there is always springiness**, so the added stiffness wins and **F1 goes up**.
  - In **Mode 2**, everything alternates, with compression near the closed end, compression further up, and **the motion of the mass in between**, right where /a/ squeezes, so the added mass wins and **F2 goes down**.
- **The vowel /a/ is the opposite of /i/**, with **F1 up and F2 down**. Students who measured their own /a/ found a high F1 and an F2 not far above it, which fits the prediction.

Section 11: The Source and How F0 Varies

- The **vocal folds** differ from person to person, and the **rules for mass and stiffness** predict their F0.
  - **Big, loose, heavy vocal folds** have **high mass and low stiffness**, so their **F0 is low** and their harmonics are packed closely together.
  - **Tighter, lighter vocal folds** give a **higher F0**, with the harmonics spaced farther apart.
  - **Very tight, light vocal folds** give an even **higher F0**, with the harmonics spaced farther apart still.
  - So a lower F0 packs **more harmonics** into the same range of frequencies, and a higher F0 spaces them **farther apart**.
- **F0 is not fixed during speech**, because only a robot keeps F0 constant, and real speech **changes F0 all the time**, in every vowel.
  - "The dog slept here" and "The dog slept here?" differ mainly in **F0**, which **goes down** at the end of the statement and **goes up** at the end of the question.
  - Speakers change F0 by **changing the stiffness of the vocal folds** with the muscles in the larynx.

- Listeners use those changes in F0 to understand meaning, and even to tell when someone is **sarcastic** or **pretending**.

Section 12: The Filter and the Core Rule

- The **air in the vocal tract** has its own formants. For **schwa**, the tube has no constriction, so its formants sit at the evenly spaced odd multiples that a uniform tube predicts.
  - The exact values vary with **how long the vocal tract is** and **how heavy and stiff the air is**, which depends on many factors, such as how wet the vocal tract is.
  - Compared with schwa, **/u/** moves F1 and F2 **both down**, **/i/** moves **F1 down and F2 up**, and **/a/** moves **F1 up and F2 down**.
- **Formants are all about where the constriction is in the vocal tract**, and they interact with the harmonics by one core rule. **A formant encourages the closest harmonics to be audible, and harmonics far from that formant are muted.**
  - Harmonics that sit near a formant get **amplified**.
  - Harmonics further from the formant get weaker, and those far away **will not be audible** to a listener.
- **The shape of the vowel is the filter, because the voiced source supplies the harmonics and the filter chooses which harmonics survive.** Formants do not make new frequencies, but instead decide which of the source's harmonics come through strongly, and each vowel's formant targets match its constriction.
  - **Schwa** has its formants evenly spread up the spectrum.
  - **The vowel /u/** has a low F1 and a low F2.
  - **The vowel /i/** has a very low F1 and a very high F2, so its first two formants are far apart.
  - **The vowel /a/** has a high F1 and a fairly low F2, so its first two formants sit close together.

Section 13: Source and Filter Together

- **This interaction between source and filter is really important.** Picture a person with **heavy, loose vocal folds**, whose harmonics are closely spaced, saying **schwa**.
  - There are **lots of harmonics near the first formant**, so the ones closest to it are **amplified**, while those farther away get weaker.
  - Between the two formants, the harmonics get **weaker**.
  - Near the second formant, the harmonics get **amplified** again.
  - So **the lower F0 is, the more harmonics the formants have to shape.**
- Now picture **very light, very stiff vocal folds**, whose harmonics are widely spaced. The first formant may have **no harmonic close by to amplify**, and **sparse harmonics can miss a formant or make it look ambiguous**, which is why some students had a hard time finding their formants in lab.
- Two everyday cases come from this mismatch.
  - **Opera singers** have such a high F0 that their **formants barely line up with any harmonics**, so listeners can hardly hear which vowels they are making, and opera is hard to understand. Singers are taught to shape their mouths so that their harmonics line up with their formants, often by opening the mouth wide like a smile.
  - **Very small children** have a very high F0, and their formants and harmonics line up so poorly that only their caretakers, who are used to their speech, can easily follow them.

- **Fo can be measured in Praat** by taking a spectral slice of a vowel and selecting the lowest harmonic. For a measured male voice, the next harmonic came out at about **double** the first, so the theory holds.

Section 14: Spectrum Versus Spectrogram

- **A spectrum captures one instant, and a spectrogram stacks many spectra over time.** Two things change in the spectrogram.
  - **Time runs along the horizontal axis, and frequency runs up the vertical axis.**
  - **How dark a spot is tells how much energy, or amplitude, there is**, so high amplitude is **black**, medium amplitude is **gray**, and low amplitude is **white**.
  - So a formant appears as a **dark band** at its frequency, and its rise and fall over time can be followed.
- Speech is **not one steady vowel**, because speakers are constantly changing sounds, so **the formants are constantly changing**. In the spectrogram of **/ia/**, the lowest band **rises** from /i/ to /a/, so F1 goes up, and the next band **falls**, so F2 goes down.
- A recording of **"Where were you a year ago?"** follows the same rules.
  - Where **F2 rises high**, the speaker is making a sound like **/i/**. The squeeze of air near two thirds raises F2 and lowers F1, so **/i/ is the classic vowel with a low F1 and a high F2**.
  - In **"you," all the formants go down**, just as /u/ predicts.
  - Across the whole phrase, the formants keep going up and down as each sound is made, so everything in the theory appears in real speech.

Section 15: The Complexity of Speech Perception

- **One experiment reveals how complex listening to speech really is.** A speaker recorded the syllables **"ma"** and **"na,"** the **m** sound was **cut out of "ma,"** the **a** vowel was **cut out of "na,"** and the two pieces were **pasted together**, so an m from "ma" was followed by an a from "na."
  - Since the consonant came from "ma," the result should sound like **"ma."** Instead, **it sounds like "na."**
- **Listeners decide which consonant they heard mostly from the vowel that follows it**, because the formants of the vowel differ depending on the consonant before it.
  - In the **a that follows m**, the formants stay **fairly stationary**.
  - In the **a that follows n**, one formant **drops sharply**.
  - That drop in a formant inside the vowel is what tells the listener "n," so it **overrides** the m that was pasted in front.
- **Speech perception does not decide sound by sound, slice by slice.** Listeners take in **a whole stretch of speech** and then decide what they heard, often deciding about a consonant **later in time**, during the vowel.
  - For decades, speech recognition machines failed because they were built to recognize each little piece separately, the way people *think* they listen.
  - **An AI that understands speech has to be complex enough to use later information to decide about earlier sounds**, and this is why large language models were needed for speech recognition.

Section 16: A Neural Network for Vowels

- A **neural network** can be **trained on formants** to **decide which vowel is being spoken**. The **W's**, which are the weights, contain the **knowledge** in a neural network, and training adjusts them until the network makes good decisions.
- This network only multiplies and adds, with **no activation functions**. It takes **two inputs**, passes them through **three hidden layers**, and ends in **five outputs**.
  - The **two inputs** are **F1 and F2**, the frequencies of the first two modes of the air's vibration, measured from spectrograms.
  - Each hidden layer **does inner products**, multiplying the incoming numbers by the W's and adding them up, and so makes a new **representation** of the input.
  - This is like information arriving at the ear and then being processed by one part of the brain after another before the brain decides what it heard.
  - The **five outputs** are **/done/, /dune/, /dean/, /Don/ and /Dan/**, with one neuron for each vowel.

Section 17: Random Weights, Forward Flow and a Bad Decision

- The example input has **a very low F1 and a very high F2**, and **a very low F1 with a very high F2 is the pattern for /i/**, so a smart network should decide **dean**.
- **Before training, every weight is a random number between negative one and one.** A positive weight pushes the next neuron up, and a negative weight pushes it down.
- In **forward flow**, the input flows forward through the network one layer at a time.
  - Each neuron in the first layer multiplies F1 and F2 by its own weights and adds the products, and **none of these calculations have to be done by hand.**
  - That gives three numbers, which pass through the next weights to give two numbers, and those pass through more weights to give four numbers.
  - Those four numbers pass through the last weights to give the **output, which is a probability distribution over the five words**.
- The output puts the **most probability on done** and spreads the rest over the other four words.
  - The values **add up to one**, so it is a **good probability distribution**.
  - But it says **"done,"** while the vowel was **/i/**. **The W's are bad**, so the network is "stupid" before training, and the result is met with an angry face.
- **Training is what fixes the bad W's.** The network is told the right answer as a distribution with all of its probability on dean and none on any other word, and **stochastic gradient descent** then changes each weight. The next time the same formants come in, the inner products should make the value for **dean** higher than all the others.

Section 18: The Same Network in PyTorch

- The same network can be written in **Python** using **PyTorch**. The code imports **torch** and **torch.nn as nn**, and it builds the network inside **nn.Sequential**, which means **a sequence of layers** that the numbers pass through in order.
- Each layer is an **nn.Linear**, which says how many numbers the layer takes in and how many it gives out.
  - The first layer, **nn.Linear(2, 3)**, says that it is a layer that takes **two inputs** and makes **three outputs**.
  - The next layer, **nn.Linear(3, 2)**, takes those **three** numbers and makes **two**.

    - The layer after that, **nn.Linear(2, 4)**, takes **two** numbers and makes **four**.
    - The last layer, **nn.Linear(4, 5)**, takes **four** numbers and makes **five**, one for each vowel.
    - Each layer's output size has to match the next layer's input size, so that the numbers can flow all the way through.
- Each **nn.Linear** holds a weight matrix with **one row for each receiving neuron** and **one column for each incoming value**. Setting **bias=False** removes the bias, an extra constant added to a neuron's sum, so each layer only multiplies and adds, just as in the diagram.
- This code is **only a declaration, with no training**, because it describes the network the way the machine sees it. It does not yet feed in any formants or change any weights.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 15
Date: Tuesday, October 27
Professor: Khalil Iskarous

Section 1: A Neural Network as a Decision Machine

- **A neural network is a numerical processing machine**, which means it takes in numbers, processes them, and **makes a decision**. This one is a very rudimentary **speech recognizer**, because it takes the **formants** measured from vowels in lab and decides **which vowel was spoken**.
  - The two **input neurons** just take information from the environment, with one taking **F1** and the other taking **F2** for one person's vowel.
  - The five **output neurons** stand for the words **done, dune, dean, Don and Dan**.
  - Between them are **three hidden layers** of neurons, which are called **hidden** because they are neither the input nor the output.
- **The weights are the network's knowledge.** Every connection between two neurons has a **weight**, a positive or negative number often written **W**, and everything the network "knows" about vowels is stored in these numbers.
  - The network is **fully connected**, so every neuron sends a connection to every neuron in the next layer.
  - The weights in the diagram are made up for teaching, and they **stay fixed** while the network works out its answer.
  - **A positive weight pushes the next neuron up, and a negative weight pushes it down**, and the **bigger** the weight, the stronger the push.
- The weights leaving the formants are **tiny**, while the deeper weights are **much larger**. The formants are big numbers in the hundreds and thousands of hertz, so tiny weights are enough to turn them into modest values for the rest of the network.
- **How many layers and neurons a network should have is not known in advance.** With **no hidden layers**, and the inputs wired straight to the outputs, the network would most likely keep making mistakes. **More numbers give the network more freedom**, so as those numbers improve, it can capture something as hard as recognizing a vowel from just two numbers, and in practice people try the simplest design first, see how it performs, and then add more.

Section 2: Forward Propagation with Inner Products

- **Forward propagation** is how the network makes a decision, with the input traveling forward through the layers one layer at a time. The example input is a vowel with a **low F1 and a high F2**. There are **no biases**, meaning no extra constants added in, and the hidden neurons are **linear**, so each one passes its sum straight on.
- **Each neuron computes an inner product.** It multiplies each incoming number by the weight on that connection and then **adds** the results, so for the top neuron of the first hidden layer, the inner product is

$$w_1 F1 + w_2 F2$$

  - Here w_1 is the weight on the connection from F1, which is small and positive, and w_2 is the weight on the connection from F2, which is small and negative.

- With the example formants, the F1 part adds a small amount and the F2 part takes away a bit more, so the result lands **just below zero**.
  - That number is the neuron's **activation**, and here it is called a **linear activation**, because nothing else is done to it.
  - **None of these calculations ever have to be done by hand**, because what matters is understanding what is happening.
- **The same thing repeats at every layer.** The activations of one layer become the inputs to the next layer, which does more inner products with its own weights, and the network keeps going through all three hidden layers until it reaches the output. Each layer has to wait for the one before it, so **a change at the input ripples all the way through**.
- **Knowledge has been reduced to positive and negative numbers, and making a decision uses nothing but multiplying and adding**, which is arithmetic from second grade.
  - The word **inference** means figuring out a decision, and almost all of its work is done by **simple numbers that hold the knowledge** together with **addition and multiplication that do the inferring**.
  - This is one of the most surprising facts about AI. Emulating human cognition might seem to need incredibly complex math, but the systems people talk to all the time use only these elementary operations.
- **This idea did not come out of nowhere.** The brain is a **chemical and electrical machine**, and over the last century or so, scientists discovered that its chemistry performs operations like **addition and multiplication**, so the operations inside a neural network are close to what the brain actually does.

Section 3: Probabilities and the Network's Decision

- After the last layer, the final numbers have to be **turned into a probability distribution**. **A probability distribution is a set of numbers that add up to one**, and those numbers are **the network's decision**.
  - Each number is how sure the network is about one vowel, so a value of one tenth means a **ten percent chance** that the formants were for that vowel.
  - **Probabilities go from zero to one, and percentages go from zero to a hundred, but they are the same idea.**
- The operation that turns the final numbers into probabilities is called **softmax**.

$$p_i = \frac{\exp(z_i)}{\Sigma_j \exp(z_j)}$$

  - Here z_i is one output's raw score, and p_i is its probability.
  - **Softmax just converts numbers into probabilities.** It makes every score positive and divides each one by the shared total, so all five add up to one.
  - **A bigger score always ends up as a bigger probability**, and if one word's score goes up, the probabilities of the other words have to go down.
  - The details of the calculation do not need to be known, because what matters is that **the probabilities add up to one** and **what they mean as a decision**.
- For the example vowel, the network gives **done a little over half of the probability**, which is **much higher than any other vowel**, so **the network decides it heard "done."** The highest probability is always the decision.

Section 4: Raising an Activation or a Weight

- The network's decision can be reasoned about without calculating anything. Take the **top neuron of the last hidden layer**, whose activation is **positive**, and its connection to **done**, whose weight is also **positive**.
- **If that activation goes up, the probability of done goes up.**
  - The activation enters done's inner product multiplied by that weight.
  - Both numbers are **positive**, so their product is positive.
  - A bigger activation makes that positive contribution **bigger**, which **raises** done's score and so raises its probability.
- **If that weight goes up, the probability of done also goes up**, for the same reason. Everything in that product is positive, so making either number bigger makes the contribution bigger.
- **Every question like this comes down to the inner product.** When two positive numbers are multiplied, the result is positive, and making either one bigger **increases** that neuron's contribution to the probability. The point is never to work out what one decimal times another comes to, but **always to see what actually happens to the decision.**

Section 5: Backpropagation and the Desired Output

- **This network is "stupid,"** and in fact **all neural networks start out stupid**, because they start with **random weights**, which means they know nothing. This one decided "done," but the formants were actually for **/i/, as in dean**.
- **A network becomes smarter only by being told the right answer**, and the right answer is given as a **desired output**.
  - The desired output for dean puts **no chance on done, dune, Don or Dan and all of the chance on dean**.
  - A target written with **zeros everywhere and a single one** is called a **one-hot encoding**.
  - In practice, nobody sits there saying "right" or "wrong." Instead the network gets a table of data, where each row says, for example, that when this person produced these two formants, the vowel was dean.
- The output the network actually produced is called the **computed output**, and the gap between it and the desired output is the **error**. Here the network gave dean only a small chance when it should have given dean all of it, so the error is large.
- **Backpropagation** is the algorithm that **changes all the weights** so that the network becomes a little smarter.
  - The goal is to **pump up the probability of the right answer** toward one and push the wrong ones down toward zero.
  - This can work because the decision is made entirely by inner products, so if the weights change, the products change, and so does the decision.
  - **Learning is gradual, just as in stochastic gradient descent.** The network does not learn the right answer all at once, but sees one example, changes its weights **a little**, sees another, changes them a little more, and so on.
- The rule for updating a weight moves it a small step in the direction that lowers the error.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

    - Here W is a weight and L is the error, also called the **loss**, and the fraction tells **whether nudging that weight up would raise or lower the error**.
    - The symbol η is the **learning rate**, which sets the size of each step.
    - **The minus sign moves each weight in whichever direction makes the error smaller.**
- The new weights are worked out **from the output backward**, so the connections into the outputs change first, then the ones into the third hidden layer, then the second, and then the first. **Forward propagation** sends information forward through inner products, while **backpropagation** sends the corrections backward, and during the backward pass the forward activations stay as they were and are reused.
    - The weight from the positive neuron into **dean** gets **bigger**, which adds more to dean.
    - The weight from that same neuron into **done** gets **smaller**, which takes away from done.
    - The weights leaving the formants **barely change**, because they use a much smaller step size to match the large values in hertz.

Section 6: Backpropagation uses the difference between prediction and target

- **Backpropagation** is the algorithm that changes all the weights so the network makes better decisions, and it is made of two algorithms. **Stochastic gradient descent** improves each weight gradually, the same way SGD improved an estimate of an average, and it has the same learning rate. **Dynamic programming** decides the order, so the last weights get improved first and the earlier ones after.
- Comparing two outputs gives the error. The **computed output** is what the network produced, putting most of the probability on done and only about 8 percent on dean. The **desired output** is the right answer, with a one for dean and zeros everywhere else.
- The difference between them is the **error**, or **loss**, and more probability on dean means less error.
- The error says exactly how each output should move, so done's probability should go down, dean's should go up, and the others should go down.
- **The weights are changed from the end backward.** First come the connections into the outputs, then into the third hidden layer, then the second and then the first, while the activations from the forward pass stay fixed and are reused.
- The weight from the positive neuron into dean gets bigger, the weights from the negative neurons into dean get smaller, and the weight from the positive neuron into done gets smaller.
- The weights leaving the formants barely move, because they use a much smaller step to match the large hertz values.

Section 7: Which Way Should a Weight Move?

- For any weight, the same thing always has to be asked. **If this weight goes up, will the error go up or down?** Comparing two marked weights makes the difference clear.
- **The positive weight from the last hidden neuron to done is an easy problem.**
    - Done's probability comes from an inner product that includes that weight times the neuron's positive activation.
    - **If the weight goes up**, that contribution grows, done's probability goes **up**, and the network gets **more wrong**, since done should have no probability at all.

    - **So the weight should go down**, because a smaller weight shrinks that contribution and pulls done's probability **down** toward zero.
    - This is a crystal-clear decision, because there is only **one inner product** between that weight and the probability.
- **The weight from F1 to the first hidden layer is a hard problem.**
    - Changing it changes the activation of its neuron in the first layer.
    - That changes the activations in the second layer, which change the third layer, which finally change the probabilities.
    - At every step, the change mixes with **many other weights and activations**, some pushing one way and some the other.
    - So **its effect on the decision is mediated by all of the calculations in between**, and it is very hard to tell just by looking whether it should go up or down, although backpropagation still works it out.

Section 8: Predict how changing a marked weight affects error

- For any weight, the question is the same. **If this weight goes up, will the error go up or down?** Two marked weights make the comparison.
- **The weight from the last hidden layer to done is an easy problem.** Done's large probability comes from an inner product that includes this weight times a positive activation of about 1.57. If the weight went up, that contribution would grow, done's probability would go up, and the network would get more wrong, since done should be zero.
- So the weight should go down, because a smaller weight shrinks that contribution and pulls done's probability down.
- There is only one inner product between this weight and the probability, so the decision is crystal clear.
- **The weight from F1 to the first hidden layer is a hard problem.** Changing it changes the activation of its first-layer neuron, which changes the second layer, which changes the third layer, which finally changes the probabilities.
- At every step, that change mixes with many other weights and activations, some pushing one way and some the other.
- Its effect on the decision is mediated by all the intermediate calculations, so just looking at it, there is no way to tell whether it should go up or down. Backpropagation still works it out.

Section 9: Why Backpropagation Starts at the End

- **Why the weights are changed from the end backward is guaranteed to be on the exam.** The answer comes from comparing the **last weights**, labeled **W5**, with the **first weights**, labeled **W2**.
- **Improving the last weights, W5, is easy.**
    - **The output probabilities are directly determined by the last weights**, and so **the error is very directly determined by the last weights** as well.
    - So the error between the computed and desired outputs can be used **directly** to decide how to change W5.
- **Improving the first weights, W2, is hard.**
    - **W2 affects its own activations, which then affect another two layers of activations along with many other weights.**

    - So it is **really hard to figure out how much blame W2 deserves** for mistakes made all the way at the end.
    - Moreover, **the effect of W2 on the errors includes the effect of W5**, since W5 is downstream of W2, and any change in W2 has to pass through W5 to reach the output.
- A strong exam answer explains that **changing an early weight's influence on the probabilities is a much more involved and mediated set of calculations, while changing a last weight changes the probabilities right away.** So the last weights pose a **simple, trivial problem**, while the early weights pose a **complex, big problem that already includes the effect of the later ones**.
- **When a problem contains smaller and smaller problems inside it, the algorithm to use is dynamic programming.**
    - **Dynamic programming** solves the **easiest, most trivial problem first**, then uses its answer to solve the next bigger one, and so on up to the hardest.
    - It comes from **Bellman** and **Viterbi**, so **backpropagation is Bellman and Viterbi's dynamic programming** applied to neural networks.
    - So **backpropagation is SGD plus dynamic programming.** SGD changes each weight **gradually**, a little at a time, and dynamic programming gives backpropagation its **"back,"** since it improves the last weights first, then the next, then the next, all the way back to the first.
- **For years people said brains could never do something as neat as a clean forward pass followed by a clean backward pass**, because brains are messy chemistry, but **that turned out to be wrong**. Research has since found that a **single brain neuron works much like a forward process followed by a backward one**, so choices that AI engineers made were already built into nervous systems long ago.

Section 10: Training Matrices

- To train the network, it has to be told **the desired output for every input**, and that information is given as **two matrices**. **A matrix is just a table of numbers**, and both tables come from the class's own lab data, with one row for each recording.
- The **input matrix, Formant_Inputs**, has one row for each recording and two columns.
    - Each row holds **one recording's two formants**, F1 and F2.
    - It has **two columns because the network takes two numbers in**.
- The **target matrix, TrueVowel_Outputs**, has one row for each recording and five columns.
    - Each row is a **one-hot answer**, with a one in the column of the true vowel and a zero everywhere else.
    - It has **five columns because the network decides among five vowels**, in the order done, dune, dean, Don and Dan.
- **The rows of the two tables must line up**, because row by row they describe the same recording.
    - If the first row of formants comes from a recording of **done**, the first row of the target has to say done.
    - The tables effectively tell the network that if it gets one pair of formants it had better say one vowel, and if it gets another pair the answer is a different vowel, and that holds for every row.

    - If one table were reordered and not the other, the network would learn from wrong answers.
- The rows on display are only a sample, not the full dataset. The network keeps its own **computed output**, but the **target never changes**, and training compares the two and changes only the **weights**.

Section 11: The Same Network in PyTorch

- A short **Python program** trains this network. It looks mysterious at first, but each part has a plain meaning, and **the code never has to be written from scratch**, because what matters is what each line **does**, not its brackets and punctuation.
- **The import lines bring in a library.**
    - The line **import torch** means **use the PyTorch library**, and **from torch import nn** brings in its neural network tools.
    - A **library** is code that someone else already wrote, so a programmer does not have to write every step. The library does all the background work, which means a network can be described in just a few lines.
    - So **these lines bring into Python everything needed for doing inner products and backpropagation**, which means anyone can build these networks without solving backpropagation themselves.
- The command **torch.load** reads the two saved tables from files, the formants and the one-hot answers. **Load** means to bring information in, which is the same thing that happens when a file is double-clicked, and **.float()** turns the values into decimal numbers.
- **The command nn.Sequential means that the network is a sequence of layers**, one layer after another. That is the most basic kind of neural network, and it describes every network built so far.
- **The command nn.Linear means a layer that does inner products by multiplying and adding.**
    - The first layer, **nn.Linear(2, 3)**, takes **two inputs** and produces **three outputs**, so it is the first hidden layer.
    - The next layer, **nn.Linear(3, 2)**, takes **three** numbers and produces **two**.
    - The layer after that, **nn.Linear(2, 4)**, takes **two** numbers and produces **four**.
    - The last layer, **nn.Linear(4, 5)**, takes **four** numbers and produces **five**, one for each vowel.
- **The network has five layers but only four Linear lines**, because the **input layer does no processing** and just receives numbers. Only layers that do inner products get a line, so the input layer appears only in the first line's statement that it takes two numbers.
- The **last lines are the only lines PyTorch needs for learning**, and their words are recognizable.
    - The word **SGD** stands for **stochastic gradient descent**, and it is given a **learning rate**, which sets the size of each step.
    - The command **zero_grad()** clears out the old leftover corrections.
    - The **loss** is **the mistake**, the gap between the computed and desired outputs, and **CrossEntropyLoss** measures it while handling the probability part itself.
    - The command **backward()** is **backpropagation**, which works out how each weight affects the mistake.

    - The command **step()** actually **changes the weights**.
    - This program takes just **one step using all of the rows** together, so it is a **minimal example** rather than a full training run.
- The program starts from **PyTorch's own default weights**, not the hand-picked ones in the diagram, so it has the same shape but different weights and would not reproduce the diagram's numbers.

Section 12: The Sigmoid and Nearly Digital Neurons

- **Neural networks did not come out of nowhere**, because their most basic features come from **discoveries about animal brains**.
    - Neuroscientists discovered that there are **neurons**, so artificial networks have neurons.
    - They discovered that **each neuron collects information from many other neurons**, and that is exactly what the **inner product** does.
- **Adrian** discovered that **neurons are nearly digital**, which means that **they either fire or they do not fire.**
    - A real neuron's activation is not some in-between number, because it is basically **off or on**.
    - The neurons in the vowel network are not like that, since their activations can be any number at all, with no limit.
- **The sigmoid** is an **activation function** that was added to neural networks to copy Adrian's discovery. A **linear** neuron just passes on its plain inner product, while the sigmoid takes that inner product and turns it into a number **between zero and one**.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

    - Here z is the inner product, and $\sigma(z)$ is the neuron's new activation.
    - **If the inner product is large and positive, the sigmoid is near one, and if it is large and negative, the sigmoid is basically zero.**
    - So a fairly negative inner product and a hugely negative one both come out almost at zero, and a fairly positive inner product and a hugely positive one both come out almost at one.
    - In this way **the sigmoid digitizes neural processing**, making artificial neurons work like real ones, close to off or close to on.
- Without this kind of limit, real neurons would have no ceiling, but **the brain's chemistry keeps neurons either off or on**, and the sigmoid copies that.
- **Animal perception reflects this S shape too. Some things that seem alike are perceived as different, and some things that are very different are perceived as the same**, and the S shape of the sigmoid produces both.
    - In the **steep middle**, two inputs that are close together come out **far apart**, so they are perceived as different.
    - Out on the **flat ends**, two inputs that are very different come out **nearly the same**, so they are perceived as the same.
- **AI is duplicating the abilities of animals.** Engineers took each discovery about animal brains and built it in, then added their own ideas, such as other functions shaped somewhat like the sigmoid. **People often say that AI has nothing to do with human cognition, but that is false**, because a big reason it works is that it copies our own abilities, using math as simple as multiplying and adding.

Section 13: Applying Sigmoid One Neuron at a Time

- The sigmoid can be added to the vowel network **one hidden neuron at a time**, keeping **the same input and every weight the same**. Each hidden neuron switches from **linear** to **sigmoid** in layer order, and **the outputs still go through softmax**, so they stay probabilities that add up to one.
- The first neuron to switch has an inner product **just below zero**, which sits on the steep middle of the curve.
  - So the sigmoid gives a value **just under one half**.
  - That means the neuron now sends a **positive** number where it used to send a **negative** one.
- **After each switch, everything that comes after it is recalculated.** Later neurons get their inputs from earlier ones, so once an earlier activation changes, **every later inner product changes too**. That is why the old linear numbers cannot simply be run through the sigmoid one at a time, since each new value has to come from the updated layer before it.
- As the neurons switch, **the network's answer moves.**
  - At first it decides **done**, with a clear majority.
  - As the first layer switches, done's lead shrinks until **done and Don are nearly tied**.
  - When the second layer switches, **Don** takes the lead, and then **Dan** does.
  - With **every hidden neuron on sigmoid**, every hidden activation is between zero and one, **Dan is on top**, **dean has risen to second place**, and **done has dropped well behind**.
- **Only the activation rule changed and no weight moved, so this is not learning.** Even so, the decision changed completely, and because **dean still is not the top answer**, the sigmoid changes how the network behaves without making it correct.

Section 14: The Same PyTorch Program with Sigmoid

- The program becomes a sigmoid network by adding **nn.Sigmoid()** after each hidden layer. **The command nn.Sigmoid means apply the sigmoid, which means make the neuron digital**, and it goes after **nn.Linear(2, 3)**, **nn.Linear(3, 2)** and **nn.Linear(2, 4)**.
- **The sigmoid changes the values but not how many there are.** A layer that produces three numbers still produces three after the sigmoid, so the next layer still fits and the network keeps exactly the same shape.
- **The last layer, nn.Linear(4, 5), has no sigmoid**, because its five outputs go to the loss, which turns them into probabilities.
  - The **sigmoid** squeezes **each hidden neuron on its own** toward off or on.
  - The **softmax** turns **all five final scores together** into a decision that adds up to one.
- The training lines stay the same as before. The program starts from PyTorch's own default weights, so it would not reproduce the diagram's exact numbers, and in lab the sigmoid can also be swapped for **other functions**, such as one called **ReLU**, to discover whether they make vowel recognition better or worse.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 16
Date: Thursday, October 29
Professor: Khalil Iskarous

Section 1: A Network That Simulates Vowel Perception

- This **neural network** simulates a very simple piece of human perception. It takes the **two formants**, F1 and F2, of an isolated vowel and **decides which vowel it is**, and its five possible answers are **done, dune, dean, Don and Dan**.
  - The **two input neurons** take F1 and F2 from the environment.
  - **Three hidden layers** of neurons process those two numbers.
  - The **five output neurons** give the decision, with one neuron for each vowel.
  - Every neuron connects to every neuron in the next layer, so the network is fully connected.
- **The weights are the network's knowledge**, and each one is a positive or negative number. A **positive** weight pushes the next neuron up, a **negative** one pushes it down, and the **bigger** the weight, the stronger the push. The weights in the diagram are made up for teaching, and they stay fixed while the network makes a decision.
- Learning works by telling the network the right answer.
  - At first **the weights are random**, so the network "doesn't know anything" and its decisions are stupid.
  - The right answer is given as a **probability distribution**, so for a recording of dean it puts all of the probability on dean, meaning the network should be **completely sure it was dean**.
  - That target is usually very different from the distribution the stupid network produces at first. The difference is a **mistake**, and the network learns by correcting its mistakes.

Section 2: Forward Propagation and the Network's Decision

- **Forward propagation** carries one input through the network to a decision. The example vowel has a **low F1 and a high F2**, and it is one person's /i/.
- **Every step is an inner product**, which means multiplying each incoming number by its weight and adding the results, so for the top neuron of the first hidden layer, the inner product is

$$w_1 F1 + w_2 F2$$

  - Here w_1 is the weight from F1, which is small and positive, and w_2 is the weight from F2, which is small and negative.
  - The F1 part adds a little and the F2 part takes away a little more, so this neuron's **activation** lands **just below zero**.
  - The other neurons do the same thing with their own weights, and layer by layer the activations become the inputs to the next inner products, until the network reaches a decision.
  - Since the two weights point in opposite directions, **a higher F1 pushes this neuron up and a higher F2 pushes it down**.
- **The last numbers are turned into a probability distribution**, a set of numbers that **add up to one**, and the operation that does this is called **softmax**.

$$p_i = \frac{\exp(z_i)}{\Sigma_j \exp(z_j)}$$

- Here z_i is one vowel's final score, and p_i is its probability.
- Softmax makes every score positive and divides each one by the total, so **a bigger score always gives a bigger probability** and all five add up to one.
- Raising one vowel's score **lowers the probabilities of the others**, since they all share the same total.

- With these weights, **done gets the highest probability, a little over half**, so **the network decides "done."** But the correct answer is **dean**, so the network's knowledge is wrong and it has to change its weights.

Section 3: Real Vowel Data Overlaps

- The vowel data from the class's own lab is plotted with **F1 along one axis and F2 along the other**, with hundreds of recordings and the same number for each vowel. **Each vowel forms a cloud of points**, since the same vowel comes out a little differently every time.
- **Some of the patterns are clear.**
  - **The vowel /i/, as in dean**, sits apart with a **high F2 and a low F1**, as the theory predicts.
  - **The vowel /ae/, as in Dan**, spreads toward **high values of F1**.
  - So /i/ is clearly different from /u/, the vowel of dune.
- **But there is a lot of overlap.**
  - **The vowels of dune, Don and done crowd into the same region**, with a middle F1 and a fairly low F2.
  - So a new speaker whose formants land in that region could be saying any of the three.
  - Some points are probably **errors**, but real data always has errors, just as talking to an AI on a train has background noise and a photo in bright sun has glare. Those points stay in the data because **that is what real life is like**.
- **This overlap is why AI is hard.** If every vowel sat in its own separate spot, recognizing vowels would be easy, but instead a network has to see the data **many times** to learn what really separates the vowels. **No setting of the weights can separate recordings that land on top of each other**, because two formants are all the network gets.
- This overlap was discovered when **Peterson and Barney** measured the vowels of many speakers from spectrograms.
  - They found an **enormous overlap** between vowels and realized that speech recognition would be far harder than anyone expected.
  - For decades, people tried to write programs to solve it, including trying to **normalize** away the differences between speakers, but **none of it worked**, and what finally worked was **a neural network trained with backpropagation**.
- **Static vowels turned out to be harder than moving speech.** For years, everyone predicted that a **static** problem like this one, with two formants from one moment, would be easy, and that **dynamic** speech, with formants always changing, would be very hard. **Everyone turned out to be wrong**, because large language models deal with very dynamic information and work much better, since they use information from before and after each moment.

Section 4: Backpropagation and the Desired Output

- **Backpropagation** is the algorithm that **changes all the weights** so that the network makes better decisions, and it is made of **two algorithms** working together.
  - **Stochastic gradient descent (SGD)** improves each weight **gradually**, in the same way that SGD improved an estimate of an average from data, and it has the same issues, including a **learning rate**.
  - **Dynamic programming** decides the **order**, so the last weights get improved first, then the ones before them, and so on back to the start.
- **Comparing two outputs gives the error.**
  - The **computed output** is what the network produced, which here puts most of the probability on done and only a small amount on dean.
  - The **desired output** is the right answer, with all of the probability on dean, and a target with a single one and zeros everywhere else is called a **one-hot target**.
  - The difference between them is the **error**, also called the **loss**. Here the loss depends on how much probability went to dean, so **more probability on dean means less error.**
- The error says exactly how each output should move. **The probability of done should go down, the probability of dean should go up, and the probabilities of the other vowels should go down.**
- Each weight is changed with the **update rule**.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

  - Here W is a weight and L is the loss, and the fraction tells **whether raising that weight would raise or lower the error**.
  - The symbol η is the **learning rate**, which is the size of each step.
  - **The minus sign moves each weight in whichever direction lowers the error.**
- **The weights are changed from the end backward**, starting with the connections into the outputs, then the connections into the third hidden layer, then the second, and then the first. The activations from the forward pass stay fixed and are reused.
  - The weight from a positive neuron into **dean gets bigger**, so it adds more to dean.
  - The weights from the negative neurons into dean **get smaller**, so they take less away from dean.
  - The weight from that same positive neuron into **done gets smaller**, which lowers the wrong answer.
  - The weights leaving the formants **barely move**, because they use a much smaller step to match the big values in hertz.

Section 5: Which Weights Are Easier to Change

- The same thing has to be asked about every weight, which is **whether the error will go up or down if this weight goes up**. The weights at the **end** of the network are much easier to change than those at the **beginning**.
- **The end weights are easy because they directly change the decision.**
  - Done's probability comes partly from multiplying a positive end weight by a positive activation, along with the other products feeding done.

    - **If that end weight went up, done's probability would go up**, which is the exact opposite of what is needed.
    - So **that end weight should go down**, and any change to an end weight is **directly, automatically and immediately** reflected in the probabilities.
- **The beginning weights are hard because their effect is buried under many layers.**
    - Changing a weight in the first layer changes that layer's **activations**, since activations are inner products of the weights with the inputs.
    - Those changed activations change the activations of the **next layer**, even with that layer's weights held fixed.
    - Those changes reach the **third layer**, and the third layer's changes finally reach the **probabilities**.
    - So **changing an early weight changes one thing, which changes another, which changes another, which finally changes the decision.** Just by looking at it, there is no way to tell which direction it should move, but backpropagation still figures it out.
- **This is the same trap as the greedy choice in dynamic programming.** Picking the cheaper of two next steps without looking ahead can lead to the wrong answer, and in the same way, a weight's local appearance says little about its effect on the final decision.

Section 6: Why Backpropagation Runs Backward

- **Dynamic programming says that if problems can be ordered by difficulty, the easiest should be solved first, then the next, and so on up to the hardest**, and backpropagation follows exactly that order.
- **The last weights, W5, are the easiest problem.**
    - **The output probabilities are directly determined by the last weights**, so **the error is very directly determined by them** as well.
    - So the error can be used **right away** to decide how to change W5.
- **The first weights, W2, are the hardest problem.**
    - **W2 affects its own activations, which then affect another two layers of activations along with many other weights.**
    - So it is **really hard to figure out how much blame W2 deserves** for a mistake made all the way at the end.
    - Moreover, **the effect of W2 on the error already includes the effect of W5**, because W5 is downstream of W2.
- **Solving the end first makes the rest easier.** Changing the last weights first teaches something that makes the problem of the next layer easier, which makes the layer after it easy too, all the way back to the start, and that is exactly dynamic programming. So **backpropagation is Bellman and Viterbi's dynamic programming combined with SGD**, where SGD improves each weight a little at a time and dynamic programming says to go **from the end backward**.
- **Backpropagation is central to the world right now**, because modern AI systems run it on enormous numbers of weights, step by step, for months. These ideas come from **physics** and are closely tied to the **dynamical systems** behind acoustics and masses and springs.

Section 7: Training Matrices and Learning from Errors

- To train the network, it gets **two matrices**, which are **tables of numbers** with one row for each student recording.
  - The **input matrix, Formant_Inputs**, has two columns, and each row holds one recording's **F1 and F2**.
  - The **target matrix, TrueVowel_Outputs**, has five columns, and each row is a **one-hot answer**, with a one for the true vowel and a zero for the others, in the order done, dune, dean, Don and Dan.
  - **Each row in one table goes with the same row in the other**, so a row of the target with all zeros and a one in the Dan column tells the network that it should be completely sure that recording is Dan.
  - The rows on display are only a sample of the full set of recordings.
- **Learning picks rows one at a time.**
  - It takes one row and uses its weights to calculate a decision for that row's formants.
  - If that decision differs from the desired answer, it **changes its weights**, the last ones first and then back toward the start.
  - It then picks another row and does the same thing again.
- **If the network already gets a row right, there is no error, so there is nothing to learn from that row.** That can be a serious problem, because a network can make the right decision **by accident** over and over and stop learning early, without ever learning the **real pattern** underneath the data.
- **Children learning English have the same problem**, as **Carol Chomsky** discovered.
  - At first, very young children say **"go"** and **"went"** correctly, because they are just **memorizing** what they hear.
  - Then they notice the real pattern, that most English verbs add **-ed** for the past, as in **jump, jumped** and **dance, danced**, and they start saying **"goed."** They have learned the rule, so they make **new mistakes**.
  - Only later do they learn that **some verbs are exceptions**, and then they get both the regular and the exceptional verbs right.
  - **Neural networks have the same issue**, because they can just **memorize** weights that give the right answers without learning the pattern, and that is what makes learning so difficult.

Section 8: The Same Network in PyTorch

- A short **Python program** builds and trains the network. **It never has to be written from scratch**, and the brackets and dots do not matter, because what matters is **what the lines do**.
- **The import lines bring in libraries.**
  - A **library** is a collection of programs that one group writes for one job and that others use without knowing how they work inside.
  - The lines **import torch** and **from torch import nn bring into Python everything needed for doing inner products, backpropagation and related work.**
  - Without libraries, the whole training program could never fit on one page.

- The command **torch.load loads** the saved tables, the formants and the one-hot answers. Loading means bringing information in, the same thing that happens when a file is double-clicked, and **.float()** makes the values decimal numbers.
- **The word Sequential means a neural network that is a sequence of layers.** All the networks built so far are a sequence of layers, while a **large language model** is a network that is **not** just a sequence of layers.
- **Each Linear line is one processing layer**, meaning a layer that has weights and does **inner products** with the incoming information. **Linear means just use inner products**, so the formulas never have to be written out.
  - The first layer, **nn.Linear(2, 3)**, says that it receives information from **two** neurons and has **three** neurons of its own.
  - The next lines take **three and make two**, take **two and make four**, and take **four and make five**.
  - The network has **five layers but only four lines**, because the **input layer does no processing** and just receives the two formants.
  - No activation function sits between the layers, so the hidden layers are **linear**.
- **The last lines are the only lines PyTorch needs for learning.**
  - The word **SGD** stands for **stochastic gradient descent**, and **lr** stands for the **learning rate**.
  - **A bigger learning rate learns a lot faster but is less accurate, while a smaller one is more accurate but learns much more slowly.**
  - The **loss** is **the mistake**, since the network calculates its mistakes and learns from them, and **CrossEntropyLoss** takes the raw scores and handles the probability part itself.
  - The command **zero_grad()** clears the old corrections, **backward()** is **backpropagation**, and **step()** changes the weights.
  - **Computing the corrections and changing the weights are separate steps**, so nothing changes until step() runs, and here all of the rows go through together for **one full step**.
- The program starts from **PyTorch's own default weights and biases**, not the hand-picked weights in the diagram, so the structure matches but the numbers would not.

Section 9: Read one full-batch training step in PyTorch

- A short **Python program** builds and trains the network, and it never has to be written from scratch, since brackets and dots do not matter. **The import lines bring in libraries**, collections of programs that other people wrote for one job, which can be used without knowing how they work inside.
- Before libraries like PyTorch existed, everyone doing AI had to write their own backpropagation code.
- The import lines import into Python everything for doing inner products, backpropagation and related work, which is why the whole training program fits on one page.
- **Sequential means a neural network that is a sequence of layers.** All the networks built so far are a sequence of layers, while a large language model is a network that is not just a sequence of layers. **Each Linear line is one processing layer**, a layer that has weights and does inner products with the incoming information.
- The first linear line says it receives information from two neurons and has three neurons, and the following lines go from three to two, two to four, and four to five.

- The network has five layers but only four lines, because the input layer does no processing.
- **The last lines are the only lines PyTorch needs for learning.** SGD is stochastic gradient descent and lr is the learning rate, here 0.1. **A bigger learning rate learns a lot faster but is less accurate, while a smaller one is more accurate but learns much slower.**
- Loss is the mistake, so the network calculates its mistakes and learns from them, and backward is backpropagation.
- Nothing changes until step is called, which is when the weights actually change.

Section 10: The Sigmoid and Nearly Digital Neurons

- **Neural networks do not come out of nowhere**, because their basic features come from **discoveries about animal brains**. Scientists found that brains have **neurons**, so networks have neurons, and they found that **each neuron collects information from many other neurons**, which is the **inner product**.
- **Adrian** discovered that **neurons are nearly digital, because they either fire or they do not fire.** A real neuron's activation is not some in-between number, because it is basically **off or on**, while the neurons of the vowel network can take any value at all.
- **The sigmoid** is an **activation function** that was added to networks to copy Adrian's discovery. It takes a neuron's **linear activation**, which is its inner product, and turns it into a number **between zero and one**.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

  - Here z is the inner product, and $\sigma(z)$ is the new activation.
  - **If the inner product is large and positive, the sigmoid is near one, and if it is large and negative, the sigmoid is basically zero.**
  - So a fairly negative inner product and a hugely negative one both come out near zero, a fairly positive one and a hugely positive one both come out near one, and only inner products close to zero land in between.
  - In this way **the sigmoid digitizes neural processing**, copying the brain chemistry that keeps real neurons close to off or on.
- **The S shape changes how differences come out.** Near zero the curve is **steep**, so a small change in the inner product makes a big change in the output, while far from zero it is **flat**, so even big changes barely matter. **Animal perception reflects this, because some things that seem alike are perceived as different and some things that are very different are perceived as the same.**
- **AI is duplicating the abilities of animals.** Engineers build in what biologists discover and then add their own ideas, so they might replace the sigmoid with other functions of a similar shape, such as **ReLU**. **People will say that AI has nothing to do with human cognition, but that is false**, because a big reason it works is that it copies our own abilities, using math as simple as multiplying and adding.

Section 11: Applying Sigmoid One Neuron at a Time

- The sigmoid is added **one hidden neuron at a time**, while the input and every weight stay the same. The neurons switch in layer order, first the ones in the first hidden layer, then the second, then the third, and the outputs still use softmax.

- The first neuron to switch has an inner product **just below zero**, which sits on the steep middle of the curve, so it comes out **just under one half**. The neuron now sends a **positive** number instead of a negative one.
- **After each switch, everything downstream is recalculated**, because later neurons depend on earlier ones. So the sigmoid network cannot be made by running the old linear numbers through the sigmoid one at a time, since each new value has to come from the updated layer before it.
- With **every hidden neuron on sigmoid**, every hidden activation sits between zero and one, and the answer moves completely. **Dan becomes the top answer**, **dean rises to second place**, and **done drops well behind**. **Only the activation rule changed and no weight moved, so this is not learning**, and even though the decision changed completely, **dean is still not the top answer**.

Section 12: The PyTorch Program with Sigmoid

- The command **nn.Sigmoid()** is added after each of the first three Linear layers. **The command nn.Sigmoid means apply the sigmoid, which makes the neurons digital**, and the layer sizes stay exactly the same, because the sigmoid changes values but not how many there are.
- **The last layer, nn.Linear(4, 5), has no sigmoid**, because its five scores go to the loss, which turns them into probabilities. The **sigmoid** squeezes **each hidden neuron on its own**, while the **softmax** turns **all five final scores together** into a decision that adds up to one.
- The training lines and the learning rate stay the same. The program starts from PyTorch's default weights, so it would not reproduce the diagram's exact numbers.

Section 13: Training from One Initialization

- Training runs **thousands of times**, which is far more than the few dozen steps needed to average a list of numbers, and **each full pass of learning is called an epoch.**
  - The starting weights are **random**, and that particular random set is called the **initialization**.
  - The network is linear, with no biases, and the formants are scaled down from hertz to kilohertz to keep the numbers small.
  - After every epoch, accuracy is measured without training. **The vowel with the largest score is the network's answer, and accuracy is the percentage of recordings it gets right**, and since these are the same recordings it learns from, this is called **training accuracy**.
- **Accuracy starts at twenty percent, which is chance.**
  - At the start the weights are random, so the network knows nothing.
  - It still is not at zero, because with **five vowels**, a random guess is right **one time in five**.
  - The word **chance** means how often someone would be right just by guessing randomly, so **with five possible answers, chance is twenty percent.**
- **After a couple hundred epochs, accuracy jumps up well above chance**, so the network is **learning something**. It dips back toward chance for a moment, recovers, and then levels off a little below half after a few hundred epochs.

Section 14: Ten Initializations and Why Accuracy Levels Off

- The same training is run **ten times**, each time starting from **different random weights**, while everything else stays identical.
- **The runs take different paths but end in the same place.**
  - All ten runs start near **twenty percent**, the level of chance.
  - Some leave chance quickly and others stay flat longer, and several briefly drop back toward chance.
  - After a few hundred epochs they **come together**, and **all ten finish at the same accuracy, a little below half**.
  - So **different starting weights change the path but not the destination**, because the learning process is so similar each time.
- **The network levels off well below perfect accuracy because the data overlap so much.** Its final accuracy is clearly better than chance, so the network has learned something, but it is still pretty low.
  - **AI is not some kind of magic.** If a person had to name the vowel for formants landing where several vowels overlap, they would have to squint hard and might still guess wrong.
  - **If the decision is very hard for humans, it is not surprising that it is hard for machines.**
  - The limit comes from **two formants not being enough to separate dune, Don and done**, not from unlucky starting weights, so trying more starting points would not help.
- **Different network designs can be tested in lab** by adding or removing layers, changing the number of neurons, or swapping in the sigmoid or other functions, and each change is checked for whether it **improves** that accuracy or makes it worse.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 17
Date: Tuesday, November 3
Professor: Khalil Iskarous

Section 1: Neural Networks from 1943 to 1986

- A **neural network** takes in information about **one moment**, uses its **weights** to process it, and makes a **decision**, and it learns those weights by **making errors and correcting them through backpropagation**. The vowel network is this kind of system, because it takes **two formants**, passes them through **three hidden layers**, and decides among **done, dune, dean, Don and Dan**.
- **The idea began with a paper by McCulloch and Pitts**, which proposed that simple mathematics could model how the brain works, and that paper described the first artificial neural network.
  - The paper starts from the **"all-or-none"** character of nervous activity, where **"all" means active and "none" means inactive**, so a neuron is either firing or not firing.
  - That is the same idea as the **sigmoid**, which pushes a neuron's activation toward off or on, so **the very first paper on artificial neural networks starts from biology about how real neurons behave.**
  - The earliest work on neural networks was about modeling how the human mind works, not about making money.
- Later, the ideas matured into the two **Parallel Distributed Processing** books.
  - The word **parallel** means that many neurons compute their activations **at the same time**, so all of the neurons in one layer work together, then all of the neurons in the next layer, and so on.
  - The word **distributed** means that the information from F1 and F2 is **spread across all of the neurons** instead of sitting in any one of them.
  - By then, **forward propagation and backpropagation** were well worked out.
- **Given some static vector of formants, these networks could make a decision about a few possible categories.** A **vector** is just a list of numbers, and **static** means one fixed list from one moment.
- What these networks lacked is that **human cognition is far more complex** than taking in one list of numbers and giving one decision at the end.
  - **Speech is incredibly time-varying**, because speakers do not produce one F1 and one F2, and **every little moment in time gives a new formant.**
  - **Language is time-varying too**, because a sentence is a whole bunch of words, one after another, and answering a question about it uses all of them.
  - So **giving a network one bunch of numbers can never model cognition that unfolds in time**, and many researchers were aware of this.

Section 2: Recurrent Neural Networks for Speech Production

- **Michael Jordan** developed **recurrent neural networks** to make sense of **speech production**, which is how the tongue, velum and lips move over time.
  - He studied features such as **nasality**, which depends on the velum going up and down, and **rounding**, which depends on the lips.

    - He realized that **a regular neural network could never handle something that complex and time-varying**.
    - So he designed a new kind of network, and that line of work **became large language models and then agentic AI**, which means that the technology at the center of modern AI was born from thinking about **speech**.
- **His network is not just sequential.** In a PyTorch network, **Sequential** means one layer after another, while Jordan's network has **two layers working in parallel**, a kind of network he invented for speech production.
    - The **plan units** take outside input and say what sequence to produce.
    - The **hidden units** combine information, and the **output units** give the response.
    - The **state units** are the new part, because they hold what the network has **just produced**.
- **The network is called recurrent because it takes its output and sends it back into part of its input.**
    - The network produces something at **one moment**, and that output loops back into the **state units**, which also feed back into themselves.
    - The network then uses it to predict the **next moment**, and uses that to predict the moment after, and so on.
    - The details of how recurrent networks work do not need to be learned, because what matters is that **cognition, and speech production especially, means understanding a whole unfolding of events in time**.
- In the network's **performance on a sequence of speech sounds**, each row is an articulatory feature, such as voicing, nasality, tongue height and rounding. The feature for **rounding rises early and stays high across many sounds**, because the outputs fed back from one moment carry into the next.
- **GPT works the same way, one word at a time.**
    - A prompt goes in, and the system predicts **the next word**, which is the first word of its answer.
    - It then feeds the prompt **plus that word** back in and predicts the next word again.
    - It keeps feeding everything back in and predicting, and people kept refining recurrent networks along these lines until **large language models** were invented.
- **Science and technology evolve from other people's ideas over long stretches of time**, as early ideas about neurons were refined into later networks and Jordan's idea was refined into large language models. The people who came up with these ideas were mostly not the ones who made a lot of money from them.

Section 3: From One Neuron to a Large Language Model

- AI systems build up in steps, and **each step extends the one before it**.
- **A single neuron takes in a list of numbers and gives back one number.**
    - A neuron takes information from all of the neurons feeding it, multiplies each input **x** by its weight **w**, and **adds them all up**, which is the **inner product**.
    - The symbol **Σ**, called sigma, just means **add them all up**.
    - This uses nothing but second-grade multiplication and addition to model the electrochemical activity of a real neuron, so **a profound idea that moves science and society can start out incredibly simple.**

- **A neural network takes in one list of numbers and gives back a whole list of decisions.**
  - Its layers of neurons are linked by sets of weights, labeled **W_1, W_2 and W_3**.
  - **Each set of weights turns one list of numbers into the next** through many inner products.
  - The last layer is a **decision vector**, with one value for each possible answer.
  - This is the kind of network that was built for the vowels, and it still takes only **one list of numbers**.
- **A large language model takes in many lists of numbers, one for each moment in a sequence, and gives back one decision about what comes next.**
  - It takes **a whole bunch of vectors**, here fifteen of them, and **each vector stands for something happening at one moment in time**, such as **one word** of a sentence.
  - In real systems the vectors number in the **hundreds of thousands to a million**, and that set is called the **context window**. Each vector is also **thousands of numbers long**.
  - The job is to **predict the sixteenth vector**, which is **a probability distribution over the next word**, with one entry for every word in the vocabulary giving the likelihood that each word comes next.
- **A large language model is made of neural networks.** Each block of **W's** is a **layer**, which is a neural network in its own right, and real systems have **hundreds of layers**. **The knowledge is still in the W's**, just as in a single network, so if an LLM writes an essay, its knowledge of how to write one is stored in its weights.
- **A large language model is also a dynamical system**, and that is why the Nobel Prize for AI was awarded in **physics**.
  - A mass-spring system is a **dynamical system**, because it starts from **initial conditions**, such as where the mass starts and how fast it is moving, and then the system itself decides what happens over time.
  - An LLM works the same way, because **its initial condition is the set of input vectors.** Each layer uses its W's to turn those vectors into **better vectors**, and then better ones again, sometimes hundreds of times, until by the last layer the vectors are good enough to **predict the next word**.
  - The mass-spring system had only **two numbers** to keep track of, while an LLM has **thousands** in each vector, but it is the same idea.
- **The calculation at every step is the inner product**, so the whole system just multiplies and adds, billions or trillions of times.
- **A large language model learns by predicting the next word and correcting its mistakes.**
  - A sentence such as **"The cat sat on the"** goes in, and the real next word is **"mat."**
  - At first the **W's are random**, so the model predicts something stupid, such as **"in,"** which is impossible, since "the" cannot be followed by "in."
  - **Backpropagation** then adjusts all of its weights so that the prediction moves closer to "mat."
  - This is repeated with an enormous number of sentences, and **surprisingly, the system ends up knowing how to speak English.** Nobody who studied human thought would

have believed that plain inner products, plus the sigmoid or functions like it, could do human language, but it works.

- **The first problem large language models solved was machine translation**, such as turning a text in English into French. People then tried the same design on speech and many other problems, and **it worked for all kinds of things.**

Section 4: Closure from Gestalt psychology

Closure is a principle from Gestalt psychology. In the figure of three broken circles, people see a whole triangle even though no triangle is drawn, because the mind fills in what is missing. The cloze procedure applies the same principle to language. It was invented as a way of measuring how readable a piece of writing is, by deleting words and asking readers to fill them in. Given "chickens cackle and blank quack", almost anyone fills in ducks right away. Filling that gap correctly requires knowing the meanings of the words, how they combine, and the structure of the whole sentence.

Section 5: Agentic Systems

- **An agentic system is a graph of large language models**, with several of them passing their output to one another, and **an agent in AI today is just an LLM.** Today's chat and programming systems work this way.
- **Each LLM in the graph gets one job.**
  - The first LLM gets the prompt with the instruction **not to write the answer but to outline it**, and that is how these systems produce long, well-structured answers.
  - The next LLM takes the outline and writes it out in **smooth sentences**.
  - Another LLM has the job of a **critic**, and its only job is to **criticize** the draft.
  - A revision LLM decides whether the draft is good enough, and if it is not, it sends the draft **back to the critic**, so there is a **loop between the critic and the reviser** that usually runs for a few rounds.
  - The last LLM produces the final output.
- **The critic is what fixed hallucination.**
  - When systems only outlined and then wrote, **hallucination**, which is the system making things up, was a huge problem.
  - Adding an LLM whose **only job is to criticize** the writer's draft is the reason there is much less hallucination now.
  - This back-and-forth is why many systems take a while to respond, and the text that sometimes flies by is the **LLMs talking to each other**.
- **The levels build up like a body.** Molecules make cells, cells make organs, and organs make a body, and in the same way **neurons make neural networks, neural networks make LLMs, and LLMs make agentic systems**, with each level inheriting the abilities of the level below it.
- **Designing these graphs is a real job**, and it does not require knowing many programming languages. Instead it requires **problem solving**, which means taking a problem in the world and **designing a graph of LLMs that will solve it**.
- **The chief insight behind all of this is Gestalt completion.**
  - In the Gestalt figure, people **see a triangle even though no triangle is drawn**, because they put the pieces together and complete it.

    - Gestalt psychologists knew long ago that **completion is a very important part of human knowledge**.
    - So **training a system only to predict the missing thing turns out to emulate human cognition.** No one would have believed that this would work, or that it would use only inner products, sigmoids and a few other simple operations.
- **Two operations run underneath everything, from one neuron all the way up to an agentic system.**
    - **Inference is inner products**, because every decision is made by multiplying and adding.
    - **Learning is SGD plus dynamic programming**, which together make backpropagation.

Section 6: Fifteen Vectors of What?

- **A large language model takes fifteen vectors, x_1 through x_{15}, and predicts x_{16}.** A vector is just a list of numbers, so **those numbers can stand for almost anything**, and whatever they stand for, the machine plays the same game of **predicting the next one**.
- The vectors can be **words**, with each vector standing for **one word** of a sentence, such as **"The cat that the dog chased ran up the old tree and sat on the ____,"** where **x_{16} is the missing next word.**
    - A speaker of English knows a likely completion, such as **"branch"** or **"mat."**
    - A new LLM starts with random W's and predicts something silly, and **backpropagation on all of its weights** moves its prediction toward a sensible word.
    - Getting it right means using **the whole sentence**, since it was **the cat**, not the dog, that sat.
- The vectors can be **slices of sound**, because the same system was applied to **spectrograms**.
    - A spectrogram is cut into **little frames**, each frame becomes one vector of **frequencies and amplitudes**, and **x_{16} is the next frame.**
    - So the system learns to **predict the next little piece of speech from all of the previous pieces**.
    - The result was **the best speech recognition ever built**, and it is also the basis for **speech synthesis** that is now very hard to tell from a human voice.
- The vectors can be **pieces of a picture**, because the same thing was done with **images**.
    - A picture is cut into **sixteen patches** read row by row, each patch becomes a vector of pixel values, and **x_{16} is the missing bottom-right patch.**
    - Trained to **predict the missing piece** across huge numbers of pictures, the system learns what a dog, a dolphin or a boat is.
    - This became the basis of modern **computer vision** and image classifiers.
- The vectors can even be **molecules**, because the same idea was applied to **proteins**.
    - A **protein** is a string of **amino acids**, which come in twenty kinds, each written as one letter, and proteins make up hair, teeth, blood and nearly everything else in the body.
    - The **insulin B chain** is read as fifteen amino acids, and **x_{16} is the next one**.
    - One of the hardest problems in science is **protein folding**, which is how a chain of amino acids folds into its particular shape, and solving folding leads to better **drug design** and **vaccines**.

    - Treating a protein as **just a sequence of amino acids** and playing the same game of predicting the next one pushed the folding problem many years ahead.
- **The large language model only ever sees vectors.** Words, sounds, pixels and molecules all reach it the same way, so one machine asks one question, which is what x_{16} will be, and it does so **with the same architecture every time**.
- **Prediction from the past has limits.** These systems predict the next thing from **past data**, so when something new happens that the past data does not cover, they can **hallucinate**, and some systems are now told to say "I don't know" instead of guessing.

Section 7: Which Sequences Can Be English Words

- English allows only **certain sequences of sounds** in its words, and knowing which sequences are allowed is called **phonotactic knowledge**.
- **A simple syllable in English can have a consonant, then a vowel, then a consonant**, which is written **CVC**, and words like **Don, mat, sit, cup, dog, fish, ring and van** follow this pattern.
    - The pattern is about **sounds, not letters**, so the **silent e** in **shave, kite and rope** does not count.
    - Two letters can also spell one consonant sound, as in **ship, chin and bath**.
- **Syllables can also have clusters**, which are groups of consonants together. A cluster **before the vowel** is called an **onset**, and a cluster **after the vowel** is called a **coda**.
    - Real onset clusters appear in **split, flat, black, glad and stop**, and real coda clusters appear in **desk, box, help, milk and lamp**.
    - Each real word has an **impossible twin**, marked with a star. The onset twins are ***sblit, *lfat, *bnack, *dlad and *tsop**, and the coda twins are ***desb, *bokp, *hepl, *mikl and *lapm**.
    - Many twins use **the same letters in the other order**, such as **flat and *lfat**, **help and *hepl**, and **lamp and *lapm**, so **the order of the sounds matters**.
    - **The order mirrors around the vowel.** In **flat**, the **l sits right next to the vowel** and the f is farther out, and in **help**, the **l again sits next to the vowel**, now on the other side, with the p farther out. In **lamp**, the **m sits next to the vowel**, and each impossible twin puts the wrong consonant against the vowel.
    - So **only certain sequences are allowed before the vowel and after the vowel.**
- **A long word is a chain of syllables**, marked with dots at the breaks, as in **ba·na·na, At·lan·tic, ath·lete and ad·mit**.
    - **The sounds t and l are fine across a syllable break, but tl cannot start a syllable**, which is why **Atlantic** is a word but ***tlantic** is impossible, and why **athlete** and **admit** work but ***thlete** and ***dmit** do not.
    - The sound **ng** can end a syllable, as in **ring**, but it can **never start one**, so ***ngapkin** is impossible.
    - Starting a word with a bad cluster, as in ***bnanana** or ***sbaghetti**, breaks the rules even when the rest of the word is fine.
    - So **whether a sequence is allowed depends on where the syllable breaks fall.**

Section 8: Guessing the Next Letter and GPT Learning

- **Phonotactic knowledge can be revealed through GPT learning.** Each item is **the start of an English word**, and the task is to **guess the next letter**. People **agree on their guesses quickly**, and **how much they agree changes from item to item**.
- **Almost anything can follow once a vowel has come.** After **ra_**, likely letters include **t, n, r and l**, and many others are possible too, because the onset is over once the vowel arrives, so the guesses **spread out**.
- **At the start of a word, the choices narrow sharply** to exactly what an English onset allows.
  - After **sp_**, the next letter is **l, r or a vowel**, and **never n**, so *spn is impossible.
  - After **k_**, the next letter is **a vowel or n**, as in **knee**, and **never t**, so *kt is impossible.
  - After **b_** and **f_**, the next letter is **r, l or a vowel**, and **never n**, so *bn and *fn are impossible.
  - After **sh_**, the next letter is **a vowel or r**, as in **shrimp**, and **never b**, so *shb is impossible.
  - After **skr_** or after **m_** at the start of a word, **only a vowel** can come, so *skrl and *mb are impossible.
  - After **bram_**, the letter **b** is almost forced, as in **bramble**, though **a new syllable** could also start there.
- **Guessing the next letter is the same as predicting x16.** The letters so far are the sequence, and the next letter is the thing to predict, which is the same **game of predicting the next token** that an LLM plays.
  - **A GPT trained only on this game learns English sounds and spelling**, because its predictions can only get good by matching the patterns of English syllables.
  - Its prediction is a **probability distribution** over the next letter, and **the shape of that distribution carries the knowledge**.
  - After **fa_**, the distribution is **flat**, spread over many letters, since almost anything can follow a vowel.
  - After **skr_**, the distribution is **sharply peaked** on a few vowels, since English allows nothing else there.
  - So **the more the letters so far restrict what can come next, the more the probability piles onto a few letters.**

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 18
Date: Thursday, November 5
Professor: Khalil Iskarous

Section 1: Building Decisions from Simple Operations

- **A neuron takes a list of inputs and combines them into one response.** Each incoming number is multiplied by its weight, and the products are added together. This **inner product** is a simple mathematical model of how inputs from other neurons contribute to a neuron's activity.
  - A **vector** is a list of numbers, so one vector supplies the inputs and one number gives the response.
  - The **weights** decide how strongly the inputs contribute. A larger contribution changes the response more strongly, so changing the weights changes what the system does with its input.
  - Multiplication and addition are elementary operations, but repeating them across many connected units creates a system with much more complex behavior. Complexity grows through the organization of simple parts.
- **A neural network combines many neurons into a decision-making system.** Every neuron in one layer produces a response, and those responses become the list of inputs received by the following layer.
  - The small network has layers of **four, three, four and five units**, linked by weight sets **W_1, W_2 and W_3**. Each set changes one list of numbers into another.
  - The final list is a **decision vector**, with a response for each possible answer. The vowel recognizer uses formants to decide which vowel was spoken, so a fixed list of measurements becomes a category decision.
  - A decision based on one fixed list captures only one part of cognition. Speech and language unfold through sequences, and understanding a sequence depends on keeping information from earlier parts available.
- **A large language model processes many vectors and predicts the following item.** In the example, fifteen input vectors pass through five layers, and the output is a prediction about the sixteenth item.
  - A **token** is an item in the sequence that the model processes or predicts. In a word example, each input vector represents a word already supplied, while the output gives probabilities for possible next words.
  - A **probability distribution** expresses which continuations are more likely. Its probabilities add up to one, so assigning more probability to one answer leaves less probability for the other possibilities.
  - The weights contain learned information that allows the system to make this prediction. Each layer changes the representations received from the layer before it, until the resulting vectors support a useful next-token decision.

Section 2: Prediction Depends on Relationships Across a Sequence

- **Knowing a language allows a person to predict a continuation from relationships that may stretch across many words.** A sentence beginning with **the dog** can contain intervening material about **my parents**, people who live far away, and other singular or plural things, before reaching the verb about the dog.

    - The suitable continuation is **barks**, because the verb belongs with the singular **dog**. If the subject were **dogs**, the corresponding continuation would be **bark**.
    - The nearest noun is therefore not enough to decide the answer. Information about the dog must remain available while the intervening words are processed, because those words do not change which noun the verb belongs with.
    - **Syntax** concerns how words are organized into sentences. Learning syntax includes learning dependencies between words, rather than learning only which word tends to occur immediately beside another.
- **An LLM learns to make a prediction by changing its weights when its answer is wrong.** A training sentence supplies both the sequence so far and the actual following word, allowing the prediction to be compared with the target.
    - If the model predicts an unsuitable word instead of **barks**, the error is used to change weights throughout its layers and heads. The changes should make the appropriate continuation more likely when the same kind of context occurs again.
    - **Backpropagation works backward from the output**, using the effects already calculated for later connections to work out the effects of earlier ones. Reusing those answers is **dynamic programming**, and **stochastic gradient descent** uses the error information to adjust the weights.
    - **Inference uses learned weights to process an input, while learning changes those weights.** Improving a prediction during training and producing an answer with the current weights are different operations.
- **Multilingual systems learn patterns from more than one language.** Learning how words go together in English and Dutch concerns related languages, while learning Japanese or Navajo involves languages with very different structures.
    - Their common problem is still predicting and organizing sequences. A multilingual system can use learned patterns across languages, while the details of the permitted word orders differ.
    - The same system can respond in different human languages instead of requiring a separate program for every language. A language model can also learn programming and database languages, because their symbols occur in structured sequences too.
- **Multimodal systems connect more than one kind of information.** Text, speech and pictures are different **modalities**, and a system can learn correspondences between them.
    - Giving a picture and receiving a caption connects visual information with language. Speech recognition connects sound with text, while speech synthesis connects text with sound.
    - The model has to represent the information in a form it can process and learn the relevant relationships. Using vectors provides a common numerical form without making pictures, sounds and words identical kinds of information.

Section 3: Layers, Heads and Better Representations

- **Predicting the next word is difficult because the needed information can be spread throughout the sequence.** A language model develops its **representations**, the information carried in its vectors, through many successive layers.
    - A later layer receives vectors already changed by the processing before it. Improving a representation means changing it in a way that helps the final prediction, rather than merely making its numerical values larger.

    - The **architecture** specifies how layers and heads are arranged, while training determines the weights. Choosing the architecture does not by itself reveal every language pattern that these learned weight combinations represent.
- **A head is a processing component that contributes to improving a representation.** Several heads work in parallel inside one layer, and their contributions are brought together before the vectors move onward.
    - Imagine asking several experts for advice on how to improve something. Each expert contributes an opinion, and combining those opinions produces an improved result.
    - The **experts are like heads**, while the **group is like a layer**. A second group adds another stage of improvement, combining new contributions to improve the result received from the preceding group.
    - The heads are described as experts because each has learned weights that contribute to processing the information. They are components within a model, rather than separate people or whole language models.
- **Attention heads consult the sequence to work out which relationships matter for the prediction.** To choose **barks**, the processing must connect the verb with **dog**, despite the intervening words about the parents.
    - Information elsewhere in the sequence can therefore matter more than the word immediately before the prediction. Consulting the sequence makes it possible to use the relevant dependency.
    - **Heads contribute in parallel within one layer, while layers supply successive stages.** The contributions within a stage are combined, then the next stage works on the representations that combination produced.
    - **For exam preparation, focus on heads and how their contributions combine within a layer.** Understanding this organization means explaining how it helps produce a useful next-token prediction.

Section 4: A Graph of Language Models

- **An agentic system connects whole LLMs into a graph, giving different models different jobs.** The graph organizes how their responses move between them, instead of using just one uninterrupted pass from the beginning of an answer to its end.
    - An **Outline model** plans the structure of a response, and a **Write model** turns that plan into connected prose. Planning first helps organize what the response will contain before its details are filled in.
    - A **Critic model** evaluates the draft, including whether its statements fit together. A **Revise model** changes it, and the revision can return to the critic for another evaluation.
    - The **critic and reviser form a loop**, while a **Final model** produces the final response. A controller can coordinate the exchange by deciding which model receives information from another.
- **The models communicate through language.** An instruction such as writing from an outline gives another model a job, just as a person can use English to specify what a collaborator should do.
    - The models can also use computer and database languages. A system can combine an English instruction with the ability to produce a program or retrieve structured information.

    - A language-teaching system with little text available can use these abilities together, organizing the available language information into material for a learner. Building a teacher involves choosing useful material and responding patiently, rather than only predicting words from a large text collection.
- **Revising a response is different from retraining the models that produce it.** In the basic agent arrangement, the individual LLMs have already been trained, and their weights stay fixed while they exchange drafts and instructions.
    - The critic's response can change what the reviser writes without changing either model's weights. The loop changes the information being exchanged and the response being produced.
    - An individual model uses the sequence already supplied as context for its next prediction. An agentic system can also use **shared memory**, a place where agents write and retrieve information, allowing information to remain available across their interactions.
    - Shared memory and the arrangement of the graph affect how the agents coordinate. These are features of the larger system, while a head is one component inside a layer of an individual LLM.

Section 5: Dynamical Systems from Oscillators to Language Models

- **A dynamical system uses a rule to turn its current state into a new state, then applies the rule again to the result.** The **state** is the set of values describing the system at a particular stage, and the starting values are its **initial conditions**.
    - The oscillator begins with **position one and velocity zero**. Because the first position update uses the old velocity, position initially stays unchanged.
    - The velocity update uses the newly calculated position. That position is positive, so velocity becomes negative, and the following position update then carries the position downward.
    - Repeatedly applying the rule develops the motion from its starting state. Predicting the future means finding the next state from the current one, then using that result again.
- **Position and velocity are updated in an alternating order.** The new position uses the old velocity, and the new velocity uses the new position, so the calculations are **interleaved**.
    - Velocity contributes to how position changes. Positive velocity raises position, while negative velocity lowers it.
    - Position contributes to how velocity changes, with positive position pushing velocity downward and negative position pushing it upward. This pulls the motion back toward the center.
    - The sequence of values forms a **trajectory**. Position follows a cosine-like pattern and velocity a shifted, minus-sine-like pattern, while the step size controls how far each update advances.
- **A recurrent neural network combines a network with feedback through time.** Jordan's network has **plan units** and **state units** feeding a hidden layer, which produces an output that is copied back into the state units.
    - The state units also have self-connections that retain a fading trace of the earlier state. The next output therefore depends on recent history as well as on the plan input.
    - The same weights are reused at successive moments. Feeding output back into input allows a repeated rule to generate a developing sequence rather than a single isolated response.

- **A language model's depth plays the role of time in this comparison.** The initial information is the set of input vectors, and each layer transforms the representations produced by the preceding layer.
  - Following one numerical value through the layers gives a changing path, while following many values gives many trajectories. Their curves are not required to be the oscillator's cosine.
  - The oscillator and recurrent network reuse the same rule across time, while each LLM layer has its own weights. What connects the examples is the dependence of a new state on the state produced before it.
  - The stock-return sequences also rise and fall over time along different paths. A dynamical description concerns how quantities develop, rather than requiring every changing system to follow one exact oscillation.
  - Feedback also occurs at the level of the agentic graph when a revised response returns to the critic. The idea of a developing state can therefore connect operations within a model with interactions between models.

Section 6: Vectors Can Represent Different Kinds of Sequences

- **Fifteen vectors can represent fifteen words, sound frames, picture patches or amino acids.** In each case the model receives **x_1 through x_{15}** and predicts the following item, **x_{16}**.
  - The words **"The cat that the dog chased ran up the old tree and sat on the ____"** form one sequence. The cat is the one that ran and sat, so a sensible next word depends on the sentence's relationships as well as on nearby words.
  - A **spectrogram** represents frequencies over time. Its fifteen successive frames supply pieces of a sound, and the prediction is the following frame, with the original frequency range running from zero to five kilohertz.
  - A picture of a house, a tree and the sun is divided into a **four-by-four arrangement of sixteen patches**. Reading row by row gives an order, with fifteen patches supplied and the missing sixteenth patch at the bottom-right corner.
  - The **insulin B-chain** example supplies **F V N Q H L C G S H L V E A L**, fifteen amino acids in an ordered chain. The following residue to be predicted is **Y**.
- **The numerical form is shared, while what the numbers represent differs.** Sound frames follow time, picture patches follow a chosen reading order through space, and amino acids follow positions along a chain.
  - The folded protein and its written sequence concern the same protein, but the predicted item here is the next amino acid. The example is about continuing an ordered sequence, rather than predicting the entire folded structure.
  - The model must have suitable representations and learning for the kind of information it processes. Calling each item a vector explains the shared input form without making the original materials interchangeable.

Section 7: Phonotactic Knowledge and Language Differences

- **Phonotactics is knowledge of which sequences of sounds a language allows.** English speakers can judge whether a possible word fits English patterns without having been explicitly taught every permitted sequence.

    - A simple syllable can have a **consonant, then a vowel, then a consonant**, as in **Don, mat, sit, fish and ring**. The pattern concerns sounds, so the silent e in **shave, kite and rope** does not add a spoken vowel.
    - Two letters can spell one consonant sound, as in **ship, chin and bath**. Sound structure and written letter structure therefore have to be distinguished.
- **Languages differ in which consonants they allow at the ends of syllables.** The Mandarin comparison focuses on nasal endings, while Cantonese also permits a final **p**, the ending that makes **cup** possible in that comparison.
    - A consonant-vowel-consonant arrangement is therefore not a complete rule for every language. Knowing that three sound positions are present does not determine which consonants a language permits in them.
    - The permitted sequences form part of a person's language knowledge. A sequence that sounds impossible in one language can fit the structure of another.
- **English allows consonant clusters whose order matters.** Clusters before the vowel occur in **split, flat, black, glad and stop**, while clusters after it occur in **desk, box, help, milk and lamp**.
    - A star marks a form unavailable in the English comparison. The onset alternatives are ***sblit, *lfat, *bnack, *dlad and *tsop**, while the coda alternatives are ***desb, *bokp, *hepl, *mikl and *lapm**. Several contain familiar consonants but arrange them in an impermissible order.
    - In **flat**, the l is next to the vowel with the f farther out. In **help**, the l is again next to the vowel, now on the other side, with the p farther out, so the order mirrors around the vowel in these examples. In **lamp**, m sits next to the vowel and p sits farther out, while ***lapm** reverses that order.
    - A sequence excluded from English can be possible elsewhere. Russian examples permit initial clusters such as **zb** or **dl**, while the Spanish comparison changes an initial **s followed by a consonant** by placing a vowel before it, as in the comparison with English **stadium**.
- **A long word is a chain of syllables, and a boundary changes which sequences are possible.** In **At·lan·tic**, t and l occur across a syllable boundary, while ***tlantic** would place tl together at the beginning.
    - The same kind of contrast separates **athlete** from ***thlete**, and **admit** from ***dmit**. The sound **ng** can finish a syllable, as in **ring**, while ***ngapkin** places it at the beginning in the English contrast.
    - Forms such as ***bnanana** and ***sbaghetti** begin with clusters unavailable in these English patterns. Familiar later syllables do not make the whole word fit, because the opening still has to follow the language's rules.

Section 8: Learning the Pattern by Predicting the Next Item

- **Predicting a word's next letter uses knowledge of its possible sound and spelling patterns.** After **ra_**, letters such as **t, n, r and l** can follow, while other beginnings allow fewer continuations.
    - After **sp_**, the possibilities include **l, r or a vowel**, while ***spn** does not fit the English pattern. After **b_** or **f_**, r, l or a vowel can follow, while ***bn** and ***fn** do not fit these word beginnings.

    - After **k_**, a vowel or **n** can follow, as in **knee**, where the written k is silent. After **sh_**, a vowel or r can follow, as in **shrimp**, while ***shb** does not fit.
    - After **skr_** or initial **m_**, a vowel follows in these patterns, while ***skrl** and ***mb** do not fit. After **bram_**, the expected b gives **bramble**, although a new syllable could also begin there.
- **A next-item predictor can learn these regularities by trying to continue the sequences in its training data.** Its probabilities express how strongly the context supports different following items.
    - A relatively open beginning permits probability to be spread over many continuations, while a restricted beginning concentrates the suitable choices. The contrast between **ra_ and skr_** connects prediction with knowledge of possible English sequences.
    - Learning syllable structure gives a smaller problem than learning how all the words of a language form sentences. The sound inventory is limited, but the permitted combinations still have to be learned.
- **A model designed to learn English word patterns uses four attention heads in each of five layers and sixty-four numbers in each token vector.** These are architecture choices, with the heads contributing within layers and the layers supplying successive transformations.
    - The initial training design uses **one thousand randomly chosen English words**. For each word, the model is intended to predict the next letter from the letters already supplied and change the weights after an error.
    - The design calls for **ten generated examples after each training step**, allowing the developing output to be inspected. The generated examples are a way to examine what the model currently produces, rather than proof by themselves that it has learned English.
    - A **pronunciation lexicon** supplies information about how words sound, allowing training to focus on pronunciation rather than treating every written letter as a separate sound. This matters for forms such as **shave**, whose final e is not separately pronounced.
- **An initial run of this model still produced largely gibberish.** Some forms may have contained vowels, but that tentative observation does not establish successful syllable learning. Correctly continuing English patterns is difficult even when human speakers find the judgments easy.
    - Training on **ten thousand words instead of one thousand**, and continuing for more iterations, are possible extensions that would give the model more opportunities to learn. These changes were proposed, but no completed successful result from them is established.
    - Architecture specifies the parts of a model, while training determines the weights that make those parts useful. Giving a model heads and layers does not by itself guarantee that it has acquired the intended language patterns.

Section 9: Programming with Language and Approximating English

- **Software 1.0 is writing a program in a programming language**, such as Python, C or Pascal. The person specifies the operations the computer should carry out.
    - **Software 2.0 learns a program through a neural network's weights.** Input and output data guide changes to the weights, so the learned mapping carries the information needed to produce outputs from inputs.

    - **Software 3.0 uses language to instruct an agentic system to construct and run a program.** The English request can specify the model, training data and desired outputs, while the system produces the code and executes the requested work.
    - **Vibe coding** is this use of natural language to direct program construction and execution. Understanding heads, layers, vectors and learning makes it possible to specify what the requested program should do, rather than asking for an unexplained result.
- **An agentic system can build and train a smaller language model from an English instruction.** The system receiving the English instruction and the model being constructed are different levels of the process.
    - The agentic system used its existing language and programming abilities to produce the word-pattern implementation. That implementation was reported to use **NumPy**, numerical Python, and a pronunciation resource.
    - The smaller model is designed to learn the word patterns through changes to its own weights. Asking an already trained system to build a model is therefore different from assuming that the new model already knows what the larger system knows.
- **Claude Shannon's 1948 paper, A Mathematical Theory of Communication, examined how sequences can approximate English.** Unrestricted letter sequences look unlike English, while adding information about the language's patterns can make generated sequences more English-like.
    - The examples progress from strings with implausible combinations toward sequences containing more familiar arrangements and recognizable pieces. Their relevance is that a language imposes structure on what can follow what.
    - Shannon's examples were statistical approximations of language, rather than modern networks with attention heads and layers. They connect information theory with sequence prediction without making the two methods the same architecture.